SCHOOL OF COMPUTING SCIENCES

DEPARTMENT OF COMPUTER APPLICATIONS

AN EXPLAINABLE DEEP LEARNING FRAMEWORK FOR IMAGE AND VIDEO DEEPFAKE DETECTION USING CNN-BILSTM-VISION TRANSFORMERS

A Project Report

Submitted to the VISTAS in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATION

BY

CHIDADHALA VENKATESH

REG NO: 24304332

Under the Guidance of

Dr. P. KAVITHA MCA, M.Phil, B.Ed , NET.,Ph.D.

MAY 2026

VISTAS

VELS

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI

SCHOOL OF COMPUTING SCIENCES

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that the main Project entitled **"AN EXPLAINABLE DEEP LEARNING FRAMEWORK FOR IMAGE AND VIDEO DEEPFAKE DETECTION USING CNN-BILSTM-VISION TRANSFORMERS" is the original record by CHIDADHALA VENKATESH, REG NO: 24304332 under my guidance and supervision for thre partial fulfilment of award of degree of MASTER OF COMPUTER APPLICATION, as per syllabus prescribed by the VISTAS.**

GUIDE **HEAD OF THE DEPARTMENT**

Submitted for the Viva-voce examination held on _____at VISTAS Pallavaram, Chennai.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Qriocity

42/3, Kannagi St, Anna Nedum Pathai, extension, Choolaimedu, Chennai, Tamil Nadu 600094

Private & Confidential

to
CHIDADHALA VENKATESH,

Subject: Offer Letter

CHIDADHALA VENKATESH,

Qriocity Ventures Private Limited is pleased to offer you the position of **"ML Developer"** based in our Chennai office. The joining date is 01-12-2025.

Kindly review the attachments carefully and communicate your acceptance of the Internship by returning to us The Employment agreement is signed.

We look forward to having you aboard.

Warm Regards ,

B. Kesy..

Kesaven B
Founder and CEO

Qriocity
Ventures Private Limited

42/3, Kannagi St, Anna Nedum Pathai, extension, Choolaimedu,
Chennai, Tamil Nadu 600094
Email - qriocityventures@gmail.com | Contact Number - 9944878589
CIN : U85499TN2023PTC164278

PROJECT COMPLETION CERTIFICATE

This is to certify that **MR. CHIDADHALA VENAKTESH** (REG NO: 24304332) from **Vels Institute of Science, Technology & Advanced Studies** has successfully completed the Project Work under the domain of Artificial Intelligence & Deep Learning at Qriocity Ventures Private Limited during the project period.

Project Title:**An Explainable Deep Learning Framework for Image and Video Deepfake Detection Using CNN-BILSTM-Vision Transformers**We appreciate your interest and dedication towards the successful completion of the project work at Qriocity Ventures Private Limited.

"Best Wishes for Your Future Endeavors"

QRIOCITY VENTURES PRIVATE LIMITED CHENNAI 600094

KESAVEN B
FOUNDER & CEO

Thanks & Regards
QRIOCITY VENTURES PRIVATE LIMITED
Chennai-94

ACKNOWLEDGEMENT

I thank the LORD almighty for his boundless blessings showered on me. If words are considered as symbols of approval and tokens acknowledgement, then the words lay heralding role of expressing my gratitude to all who have helped me directly and indirectly during my research work.

I express my sincere and profound thanks to **Dr. ISHARI K.GANESH**, Founder-chancellor, VISTAS, Chennai, thankful to **Dr. A. JOTHI MURUGAN**, Pro Chancellor – Planning & Development, VISTAS, Chennai, **Dr. ARTHI K. GANESH**, Pro Chancellor – Academics, VISTAS, Chennai, **Dr. PREETHAA K. GANESH**, Vice President – VELS group of Institutions, for their support in completing my research work.

I take this opportunity to express my sincere gratitude and thanks to our Pro-Chancellor (SOE) **Dr. M. BHASKARAN,** VISTAS, Vice-Chancellor **Dr. T. SASIPRABHA,** VISTAS and Registrar **Dr. M. CHANDRASEKARAN**, VISTAS, providing me with an opportunity to do my research work.

I would also like to thank **Dr. A. UDHAYAKUMAR**, Controller of Examinations, VISTAS, for permitting to carry the research. Their support and encouragement enabled me to complete my research work successfully.

I thank to **Dr. R. PRIYA ANAND**, Professor and H.O.D., Department of Computer Applications, VISTAS, Pallavaram, Chennai, for the guidance throughout the discussions, which gave me the driving force to pursue my research work with necessary, zeal and zest. I am grateful to her and she has helped me to move forward in my career and life.

I am greatly indebted to my guide **Dr. P. KAVITHA,** Assistant Professor, Department of Computer Applications-PG, School of Computing Sciences, VISTAS, Pallavaram, Chennai, for her able guidance and constant motivation during my research work. Her wise academic advice and ideas have played an extremely vital role in this research work. Without her support, this research work would not have been possible.

I would like to take this opportunity to thank all the teaching and non-teaching staff members of VISTAS. I also thank all my well-wishers, who helped me either directly or indirectly during my Project work.

CHIDADHALA VENAKTESH

DECLARATION

I'm, CHIDADHALA VENKATESH (Reg No:24304332) declare that the Main Project entitled "AN EXPLAINABLE DEEP LEARNING FRAMEWORK FOR IMAGE AND VIDEO DEEPFAKE DETECTION USING CNN-BILSTM-VISION TRANSFORMER" is a record of original work done by me in partial fulfilment of the requirements for the award of the degree of Master of Computer Application under the guidance of Dr. P. Kavitha for the academic year 2025 - 2026. This project work has not formed the basis for the award of any degree

(Signature of the Candidate)

An Explainable Deep Learning Framework for Image and Video Deepfake Detection Using CNN-BILSTM-Vision Transformers

Abstract

The rapid advancement of deep learning has significantly improved the realism of deepfake content, posing serious threats to digital media authenticity and security. This paper proposes an explainable deep learning framework for robust detection of image and video deepfakes by integrating Convolutional Neural Networks (CNN), Bidirectional Long Short-Term Memory (BiLSTM), and Vision Transformers (ViT). The CNN component is employed for spatial feature extraction, capturing fine-grained visual artifacts, while the BiLSTM module models temporal dependencies across video frames to detect inconsistencies over time. The Vision Transformer further enhances global feature representation by learning long-range contextual relationships within the data. To improve transparency and trustworthiness, the framework incorporates explainability techniques such as attention visualization and feature attribution, enabling interpretation of model decisions. Experimental results demonstrate that the proposed hybrid architecture outperforms existing methods in terms of accuracy, robustness, and generalization across multiple deepfake datasets. The integration of explainable AI ensures that the system not only achieves high detection performance but also provides insights into the underlying decision-making process, making it suitable for real-world forensic and security applications.

TABLE OF CONTENT

COMPANY INTRODUCTION

i. Company Profile

Overview

Qriocity Pvt. Ltd. is a Chennai-based technology and innovation company established with a clear vision — to bridge the gap between curiosity and practical, real-world solutions. The company's name itself is a reflection of its identity, drawing from the word "curiosity" to capture the spirit that drives everything it builds and delivers.

Since its inception, Qriocity has grown into a trusted technology partner for businesses, startups, and institutions looking for smart, scalable, and modern digital solutions. With a team of experienced developers, designers, and technology strategists, the company continues to redefine how technology can serve people and organizations effectively.

Core Competencies

Qriocity's expertise spans across multiple domains of modern technology. The company's core competencies lie in software development, product design, digital transformation, artificial intelligence, and technology consulting. With a talented team of engineers, UX designers, data scientists, and system architects, Qriocity specializes in building intelligent platforms that streamline operations, enhance user experience, and promote digital transformation across various sectors.

Services and Solutions

Qriocity offers a comprehensive range of technology services designed to address the evolving digital needs of its clients. These include:

- **Custom Software Development —** End-to-end development of robust, scalable applications tailored precisely to a client's business requirements and long-term goals.
- **Product Design & UX —** Thoughtfully designed, user-first interfaces that prioritize both aesthetic quality and functional clarity to deliver outstanding digital experiences.

- **Digital Transformation —** Strategic guidance and hands-on implementation to help traditional organizations transition confidently into the digital landscape.
- **AI & Automation Solutions —** Intelligent systems powered by artificial intelligence that automate complex workflows, support data-driven decisions, and enhance overall efficiency.
- **Tech Consulting & Strategy —** Expert advisory services that help clients identify the right technology approach, avoid common pitfalls, and execute their vision with clarity.

Approach and Philosophy

What sets Qriocity apart is its deeply client-centered approach. Before a single line of code is written, the team invests time in truly understanding what the client needs, why they need it, and what success looks like for them. This principle ensures that every solution delivered is not just technically strong, but meaningfully aligned with the client's objectives.

Qriocity values transparency and honest communication above all. There are no vague promises or technical jargon — only clear, consistent, and accountable partnerships. The company is not interested in one-off projects; it aims to be a long-term technology partner that evolves alongside its clients as their business grows and changes.

Core Values

The work culture and client relationships at Qriocity are built on a foundation of values that shape how the company thinks, builds, and delivers:

- **Curiosity —** A relentless drive to question, explore, and discover better ways of solving problems, which forms the very essence of the company's identity.
- **Integrity —** A commitment to honesty, transparency, and accountability in every interaction, every promise, and every deliverable.
- **Empathy —** A genuine effort to understand the people behind every project — whether they are clients, end users, or teammates.

- **Excellence —** An uncompromising standard of quality that refuses to settle for anything less than the best possible outcome.
- **Collaboration —** A firm belief that the most impactful solutions are always built together, through open dialogue, mutual respect, and shared purpose.

Our Team

Behind Qriocity is a diverse and highly capable team of engineers, designers, strategists, and innovators who come from varied backgrounds but share one defining quality — a genuine passion for building things that matter. The team brings together technical depth, creative thinking, and a human-first mindset that shows in every product they deliver.

Qriocity invests deeply in its people, fostering a culture of continuous learning, open collaboration, and personal growth. The company believes that a team that genuinely loves what they do will always go above and beyond for the clients they serve.

Headquarters

Qriocity Pvt. Ltd. is proudly headquartered in Chennai, Tamil Nadu — one of India's most vibrant and fast-growing technology centres. Chennai's strong engineering culture, deep talent pool, and expanding startup ecosystem make it an ideal base for a company built on innovation and ambition. Being rooted in Chennai gives Qriocity the advantage of working with top local talent while maintaining the standards and outlook of a globally competitive firm.

Looking Ahead

Qriocity Pvt. Ltd. remains committed to its founding mission — turning curiosity into innovation and innovation into lasting impact. As technology continues to evolve at a rapid pace, Qriocity is determined to remain at the forefront, continuously investing in new capabilities, expanding its service offerings, and deepening its partnerships with clients across industries.

The company looks forward to collaborating with organizations that share its belief in the power of technology to transform, simplify, and elevate human experiences. With curiosity as its compass and excellence as its standard, Qriocity is ready to build what comes next.

ii. Project Profile

The project titled *"***An Explainable Deep Learning Framework for Image and Video Deepfake Detection Using CNN-BiLSTM-Vision Transformers***"* focuses on addressing the serious challenges posed by the rapid growth of deepfake technology. With advancements in artificial intelligence, it has become easier to create highly realistic fake images and videos that can mislead people and spread misinformation. This creates major concerns in areas such as social media, digital security, journalism, and forensic analysis. The main objective of this project is to develop an efficient and intelligent system that can accurately detect deepfake content in both images and videos while also providing clear explanations for its predictions.

The system is developed using a combination of modern programming languages and technologies, including Python, Java, Node.js, MongoDB, React, and JavaScript. Python is used as the core language for implementing the deep learning models due to its powerful libraries and frameworks for machine learning and image processing. The backend of the system is handled using Java and Node.js, which manage server operations, API communication, and the interaction between the frontend and the model. MongoDB is used as the database to store uploaded media files, detection results, and user-related data because of its flexibility in handling large and unstructured datasets. The frontend is developed using React and JavaScript, providing a user-friendly and interactive interface where users can upload images or videos and view the detection results easily.

The proposed framework integrates multiple deep learning techniques to improve accuracy and reliability. Convolutional Neural Networks (CNN) are used to extract spatial features from images and video frames by identifying visual patterns, textures, and artifacts that indicate manipulation. Bidirectional Long Short-Term Memory (BiLSTM) networks are used to analyze temporal information in videos, helping the system detect inconsistencies in motion and facial expressions across frames. In addition, Vision Transformers (ViT) are incorporated to capture global relationships within the data, allowing the model to better understand complex patterns and improve detection performance across different types of deepfake content.

An important feature of this project is its focus on explainability. Unlike traditional deep learning systems that act as black boxes, this framework provides visual explanations for its predictions. Techniques such as Grad-CAM and attention maps are used to highlight specific regions in an image or video frame that influenced the model's decision. This helps users

understand why a piece of media is classified as fake or real, thereby increasing trust in the system. Explainability is particularly important in fields such as digital forensics and security, where decisions must be transparent and justifiable.

The workflow of the system begins when a user uploads an image or video through the React-based interface. The input is processed by the backend, where necessary preprocessing steps such as resizing, normalization, and frame extraction (for videos) are performed. The processed data is then sent to the deep learning model implemented in Python for analysis. Once the model generates the prediction, the result along with the explanation is sent back to the frontend and displayed to the user. All relevant data is stored in MongoDB for future reference and analysis.

To ensure the effectiveness of the system, the model is trained and evaluated using standard deepfake datasets. Performance is measured using metrics such as accuracy, precision, recall, and F1-score. The combination of CNN, BiLSTM, and Vision Transformers enables the system to achieve high performance in detecting both image-based and video-based deepfakes. The system is also designed to be scalable, allowing it to handle multiple users and large volumes of data efficiently.

Key Technologies and Features Include:

- Python for developing deep learning models
- Java and Node.js for backend development and API handling
- MongoDB for storing images, videos, and results
- React and JavaScript for building the user interface

Core Features:

- CNN for extracting image features
- BiLSTM for analyzing video sequences
- Vision Transformers (ViT) for global pattern recognition
- Explainable AI using Grad-CAM and attention maps
- Real-time image and video deepfake detection

- User-friendly interface for easy interaction
- Scalable system for handling large data
- Performance evaluation using accuracy, precision, recall, and F1-score

CHAPTER-01
INTRODUCTION

1.1 Background of the Study

The rapid advancement of artificial intelligence and deep learning has significantly transformed digital media generation. One notable development is deepfake technology, which leverages neural networks to create highly realistic manipulated images and videos. These synthetic media are often indistinguishable from real content, making them powerful yet potentially dangerous tools. While deepfakes have useful applications in entertainment, filmmaking, and virtual simulations, their misuse has become a growing concern in today's digital world.

1.2 Problem Statement

The increasing sophistication of deepfake generation techniques has made it difficult to detect manipulated media using traditional methods. Existing detection approaches often rely on handcrafted features or focus only on spatial or temporal aspects, resulting in reduced effectiveness. Furthermore, many deep learning models lack transparency, functioning as black boxes without providing explanations for their predictions. This creates challenges in trust, especially in critical domains like digital forensics and cybersecurity.

1.3 Objectives of the Study

The main objective of this research is to develop an efficient and explainable deep learning framework for detecting deepfake images and videos. The specific objectives include:

- To design a hybrid model combining CNN, BiLSTM, and Vision Transformers
- To extract both spatial and temporal features for improved detection accuracy
- To enhance model performance using attention mechanisms
- To incorporate explainable AI techniques for interpretability and transparency

1.4 Proposed Approach

This work introduces a hybrid architecture that integrates multiple deep learning techniques. Convolutional Neural Networks (CNN) are used for extracting spatial features from images, while Bidirectional Long Short-Term Memory (BiLSTM) captures temporal dependencies in video sequences. Vision Transformers (ViT) further enhance the system by modeling global relationships using attention mechanisms. The integration of these components ensures a comprehensive analysis of both images and videos.

1.5 Significance of the Study

Deepfake detection plays a crucial role in maintaining the authenticity and reliability of digital content. This study contributes to the field by providing a robust and explainable solution that improves detection accuracy while ensuring transparency. The proposed system

can be applied in areas such as social media monitoring, digital forensics, cybersecurity, and media verification.

1.6 Scope of the Study

The scope of this research includes the detection of deepfake images and videos using deep learning techniques. The study focuses on analyzing visual and temporal features and implementing explainability methods to interpret model decisions. However, it is limited to the datasets used for training and evaluation and may require further optimization for real-time large-scale deployment.

1.7 Organization of the Report

This report is organized into several chapters. Chapter 1 presents the introduction and overview of the study. Chapter 2 discusses the literature review and related work. Chapter 3 explains the proposed methodology and system design. Chapter 4 presents the results and performance analysis. Finally, Chapter 5 concludes the study and suggests future research directions.

CHAPTER-02

SYSTEM ANALYSIS

2.1 Introduction

System analysis is a crucial phase in the development of any intelligent system, as it involves understanding the problem, evaluating existing solutions, and defining system requirements. In the context of deepfake detection, system analysis focuses on identifying the limitations of current detection techniques and establishing the need for a more robust, accurate, and explainable framework. This chapter provides an overview of the existing systems, highlights their drawbacks, and presents the proposed system along with its advantages and feasibility.

2.2 Existing System

Existing deepfake detection systems primarily rely on traditional machine learning or standalone deep learning models. Many approaches use Convolutional Neural Networks (CNNs) to detect spatial inconsistencies in images, such as unnatural textures, lighting mismatches, and facial distortions. Some advanced methods incorporate Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks to analyze temporal variations in videos.

However, these systems often focus on either spatial or temporal features, rather than combining both effectively. Additionally, most existing models lack interpretability and operate as black boxes, making it difficult to understand their decision-making process.

2.3 Limitations of Existing System

The current systems suffer from several limitations:

- Inability to effectively capture both spatial and temporal features simultaneously
- Reduced performance on highly realistic and high-quality deepfakes
- Lack of generalization across different datasets and manipulation techniques
- Absence of explainability, leading to low transparency and trust
- High false positive and false negative rates in complex scenarios

2.4 Proposed System

To overcome these limitations, the proposed system introduces a hybrid deep learning framework that integrates CNN, BiLSTM, and Vision Transformers (ViT). The system is designed to analyze both images and videos by combining spatial feature extraction, temporal sequence modeling, and global attention mechanisms.

CNN is used to extract detailed spatial features from input frames. BiLSTM captures temporal dependencies by analyzing sequences of frames in both forward and backward directions. Vision Transformers enhance the model by learning global contextual relationships using attention mechanisms. Additionally, Explainable Artificial Intelligence (XAI) techniques are incorporated to provide visual and interpretable insights into the model's predictions.

2.5 Advantages of Proposed System

The proposed system offers several advantages:

- Improved accuracy through hybrid model integration
- Effective analysis of both spatial and temporal features
- Enhanced robustness against advanced deepfake techniques
- Better generalization across diverse datasets
- Increased transparency using explainable AI methods
- Reduced error rates compared to traditional approaches

2.6 Feasibility Study

2.6.1 Technical Feasibility

The system is technically feasible as it utilizes well-established deep learning frameworks and tools. The required technologies, such as Python, TensorFlow/PyTorch, and GPU-based computation, are readily available and widely supported.

2.6.2 Economic Feasibility

The proposed system is cost-effective since it relies on open-source software and publicly available datasets. The main cost involves computational resources, which can be optimized using cloud platforms or shared GPU environments.

2.6.3 Operational Feasibility

The system is user-friendly and can be integrated into real-world applications such as social media platforms, surveillance systems, and digital forensic tools. With proper interface design, it can be easily used by non-technical users as well.

2.7 System Requirements

2.7.1 Hardware Requirements

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- GPU: NVIDIA GPU (for faster training and inference)

- Storage: 256 GB or higher

2.7.2 Software Requirements

- Operating System: Windows/Linux
- Programming Language: Python
- Libraries: TensorFlow / PyTorch, OpenCV, NumPy, Pandas
- Development Tools: Jupyter Notebook / VS Code

2.8 Summary

This chapter analyzed the existing deepfake detection systems and identified their limitations. A hybrid and explainable deep learning framework has been proposed to address these challenges. The feasibility and requirements discussed confirm that the system can be effectively implemented in real-world scenarios.

CHAPTER-03

SYSTEM REQUIREMWNT

3.1 Hardware Requirements

- **Processor** : Intel Core i5 or higher (or equivalent AMD processor)
- **RAM** : 8 GB or more
- **Storage** : SSD with at least 256 GB of storage capacity (for faster read/write speeds)
- **Display** : Monitor with a minimum resolution of 1920 × 1080 pixels

3.2 Software Requirements

- **Programming Language** : Python 3.8
- **Database** : MySQL
- **Operating System** : Windows OS
- **Server** : WAMP Server
- **Frameworks** : Flask 1.2
- **Libraries** : TensorFlow, Scikit-learn, Pandas, NumPy, Matplotlib, Biopython

3.3 Software Description

3.3.1 Python 3.8

Python is a high-level, interpreted, and general-purpose programming language known for its simplicity and readability. It supports multiple programming paradigms, including object-oriented and procedural programming, making it highly flexible for various applications. Python was developed by Guido van Rossum and has become one of the most widely used programming languages in the world.

Python is extensively used in fields such as web development, data science, artificial intelligence, and machine learning. Its simple syntax and vast ecosystem of libraries make it ideal for both beginners and professionals. Python programs are generally shorter and more readable compared to other languages like Java or C++, which enhances productivity and maintainability.

The major strength of Python lies in its rich set of libraries and frameworks that support:

- Machine Learning and Deep Learning
- Web Development (Django, Flask)
- Image Processing (OpenCV, Pillow)
- Data Analysis (Pandas, NumPy)
- Scientific Computing and Visualization
- Automation and Scripting

Due to these advantages, Python is widely adopted by major technology companies such as Google, Amazon, Facebook, and Netflix.

3.3.2 TensorFlow

TensorFlow is an open-source machine learning framework developed by Google. It provides a comprehensive ecosystem of tools, libraries, and community resources that enable developers to build and deploy machine learning models efficiently.

TensorFlow supports both high-level and low-level APIs, making it suitable for beginners as well as advanced users. It allows easy deployment of models across multiple platforms, including servers, cloud environments, mobile devices, and web browsers. TensorFlow is widely used for building deep learning models, including neural networks for image processing, natural language processing, and predictive analytics.

3.3.3 Keras

Keras is a high-level deep learning API that runs on top of TensorFlow. It is designed to simplify the process of building and training neural networks, enabling rapid experimentation. Keras provides built-in support for:

- Convolutional Neural Networks (CNNs) for image processing
- Recurrent Neural Networks (RNNs) for sequence modeling
- Hybrid architectures combining multiple models

It offers a user-friendly interface and allows seamless execution on both CPU and GPU. Keras is widely used for prototyping and developing deep learning models efficiently.

3.3.4 Pandas

Pandas is an open-source data analysis and manipulation library built on top of Python. It provides powerful data structures such as DataFrames, which allow efficient handling of structured and labeled data.

Pandas supports data import from various formats such as CSV, Excel, JSON, and SQL databases. It enables operations like data cleaning, transformation, merging, and aggregation. Pandas plays a crucial role in preprocessing datasets before feeding them into machine learning models.

3.3.5 NumPy

NumPy (Numerical Python) is a fundamental library for numerical computing in Python. It provides support for large multidimensional arrays and matrices along with a collection of mathematical functions to operate on them.

NumPy is highly efficient and forms the backbone of many scientific computing libraries. It is widely used for performing mathematical and statistical operations required in machine learning and deep learning applications.

3.3.6 Matplotlib

Matplotlib is a comprehensive visualization library used for creating static, animated, and interactive plots in Python. It allows developers to visualize data through graphs such as line charts, bar graphs, histograms, and scatter plots.

Matplotlib plays an important role in analyzing model performance and presenting results in a clear and interpretable manner.

3.3.7 Scikit-learn

Scikit-learn is a machine learning library built on top of NumPy and SciPy. It provides a wide range of algorithms for classification, regression, clustering, and dimensionality reduction.

It is widely used for tasks such as model training, evaluation, and validation. Scikit-learn also offers tools for data preprocessing, feature selection, and performance metrics.

3.3.8 MySQL

MySQL is a widely used open-source relational database management system. It uses Structured Query Language (SQL) for managing and manipulating data stored in databases.

MySQL is known for its speed, scalability, and ease of use. It supports various database operations such as data insertion, updating, deletion, and retrieval. It is commonly used in web applications to store user data and system information.

3.3.9 WAMP Server

WAMP Server is a Windows-based web development environment that includes Apache, MySQL, and PHP. It enables developers to build and test web applications locally.

WAMP provides an easy-to-use interface and simplifies database management using tools like phpMyAdmin. It is widely used for developing dynamic web applications.

3.3.10 Bootstrap 4

Bootstrap is a free and open-source front-end framework used for designing responsive and mobile-first web applications. It includes pre-designed HTML, CSS, and JavaScript components.

Bootstrap ensures cross-browser compatibility and responsive design across various devices such as desktops, tablets, and smartphones. It simplifies the process of creating visually appealing user interfaces.

3.3.11 Flask

Flask is a lightweight web framework written in Python. It provides tools and libraries for building web applications quickly and efficiently.

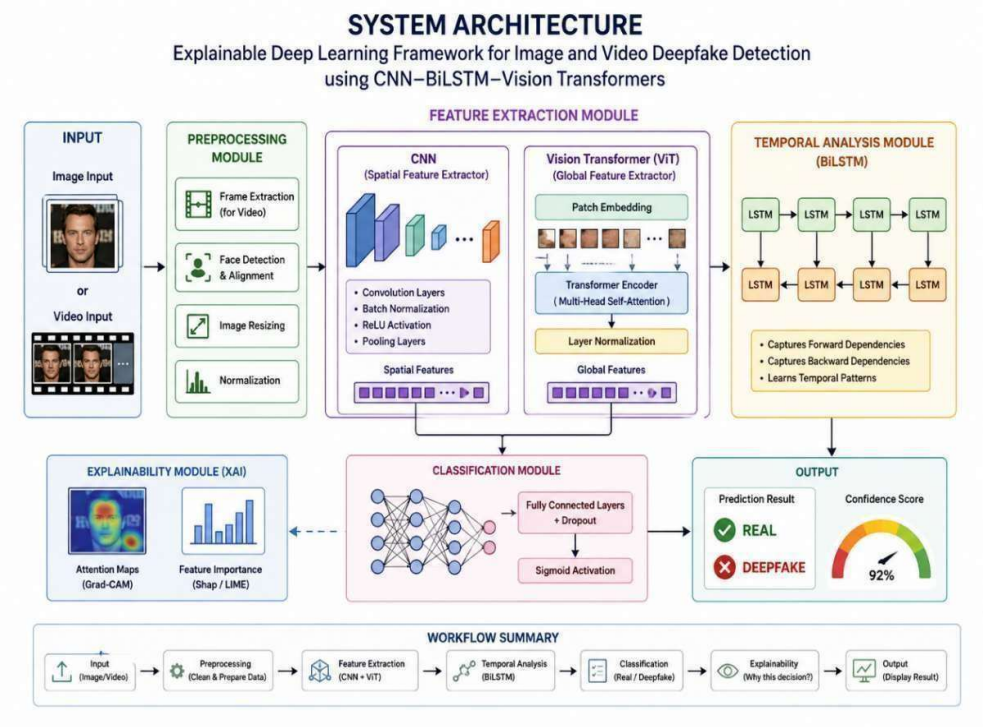
Flask follows a minimalistic approach, allowing developers to add only the components they need. It is highly flexible and supports extensions for database integration, authentication, and API development. Flask is widely used for developing machine learning-based web applications due to its simplicity and scalability.

3.4 Summary

This chapter outlined the hardware and software requirements necessary for implementing the proposed deepfake detection system. It also provided a detailed description of the technologies and tools used, highlighting their roles and importance in the development process. The combination of these technologies ensures efficient system performance, scalability, and ease of deployment.

CHAPTER-04
SYSTEM DESIGN

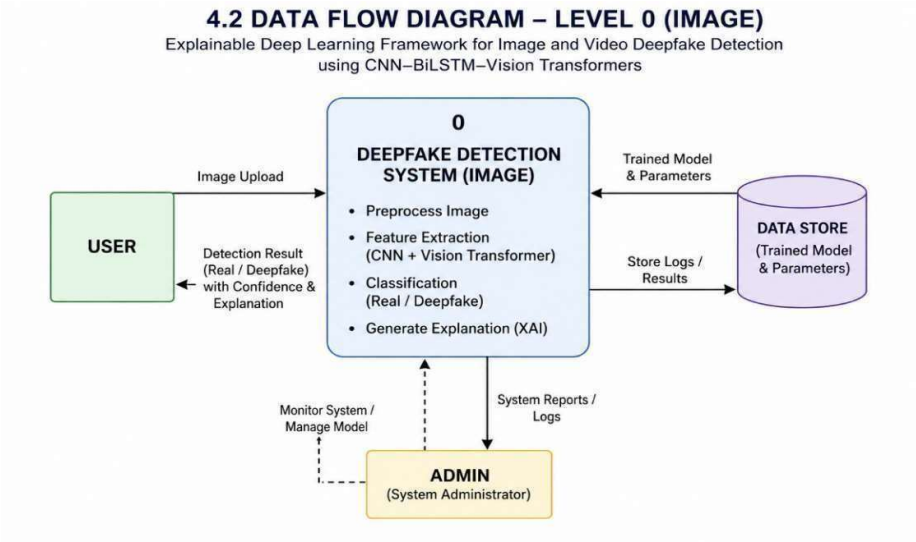
4.1. SYSTEM ARCHITECTURE



(fig:4.1. System Architecture of feature extraction module)

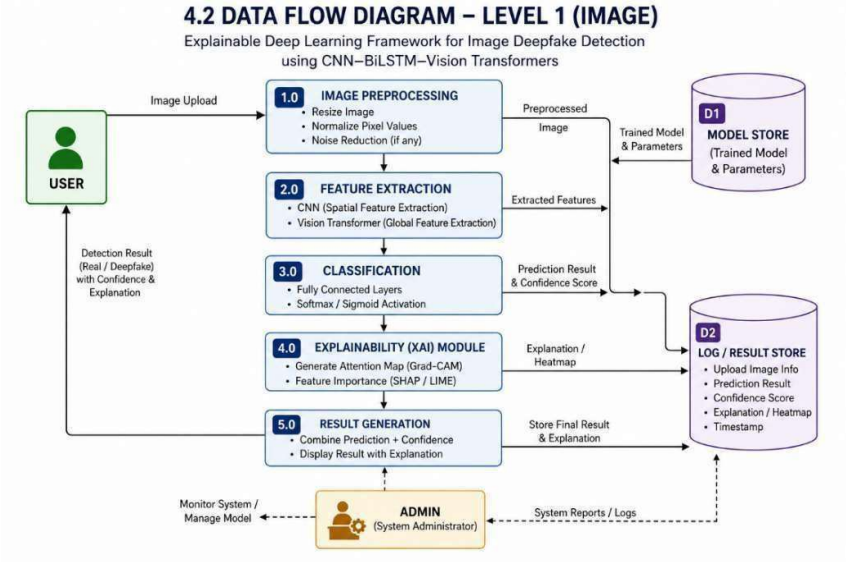
4.2. DATA FLOW DIAGRAM

LEVEL 0



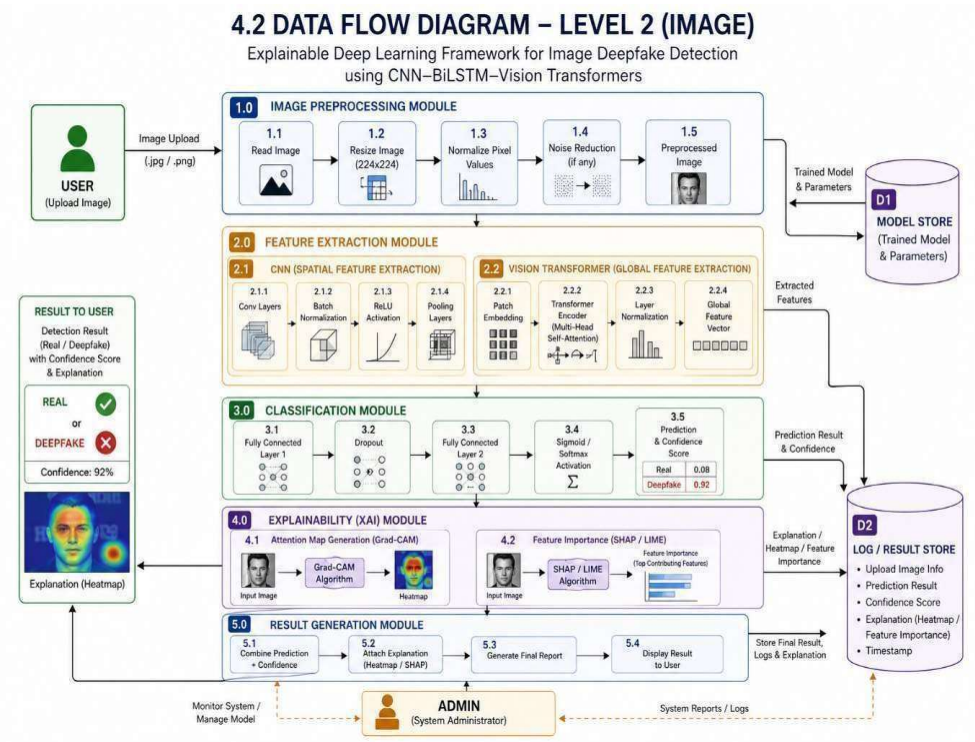
(**fi**g: 4.2. Data Flow Diagram of [level 0] DDS)

LEVEL 1



(fig: level 1 of data flow diagram)

LEVEL 2



(fig: level 2 of data flow diagram)

4.3. DATA BASE DESIGN

4.3.1 Introduction

Database design plays a crucial role in storing, managing, and retrieving data efficiently in the deepfake detection system. The database is used to maintain user information, uploaded image details, prediction results, confidence scores, and explainability outputs such as heatmaps and feature importance. A well-structured relational database ensures data consistency, scalability, and efficient query processing.

4.3.2 Database Overview

The system uses a relational database (MySQL) to store structured data. The database is designed to support:

- User management
- Image upload tracking
- Prediction and classification results
- Explainability outputs (heatmaps, feature importance)
- System logs and reports

4.3.3 Entity Description

1. User Table

Stores details of users interacting with the system.

Field Name	Data Type	Description
user_id	INT (PK)	Unique user identifier
username	VARCHAR	Username
email	VARCHAR	User email
password	VARCHAR	Encrypted password
created_at	TIMESTAMP	Account creation date

2. Image Table

Stores information about uploaded images.

Field Name	Data Type	Description
image_id	INT (PK)	Unique image ID
user_id	INT (FK)	Linked user
image_path	VARCHAR	Stored file path
upload_time	TIMESTAMP	Upload timestamp

3. Prediction Table

Stores classification results of images.

Field Name	Data Type	Description
prediction_id	INT (PK)	Unique prediction ID
image_id	INT (FK)	Linked image
result	VARCHAR	Real / Deepfake
confidence_score	FLOAT	Prediction confidence
predicted_at	TIMESTAMP	Prediction time

4. Explainability Table

Stores explainability outputs such as heatmaps and feature importance.

Field Name	Data Type	Description
explain_id	INT (PK)	Unique ID
prediction_id	INT (FK)	Linked prediction

Field Name	Data Type	Description
heatmap_path	VARCHAR	Path to heatmap image
feature_data	TEXT	Feature importance values

5. Admin Table

Stores administrator details.

Field Name	Data Type	Description
admin_id	INT (PK)	Unique admin ID
username	VARCHAR	Admin username
password	VARCHAR	Encrypted password

6. Logs Table

Stores system logs and activity records.

Field Name	Data Type	Description
log_id	INT (PK)	Unique log ID
user_id	INT (FK)	Associated user
activity	TEXT	Action performed
timestamp	TIMESTAMP	Time of activity

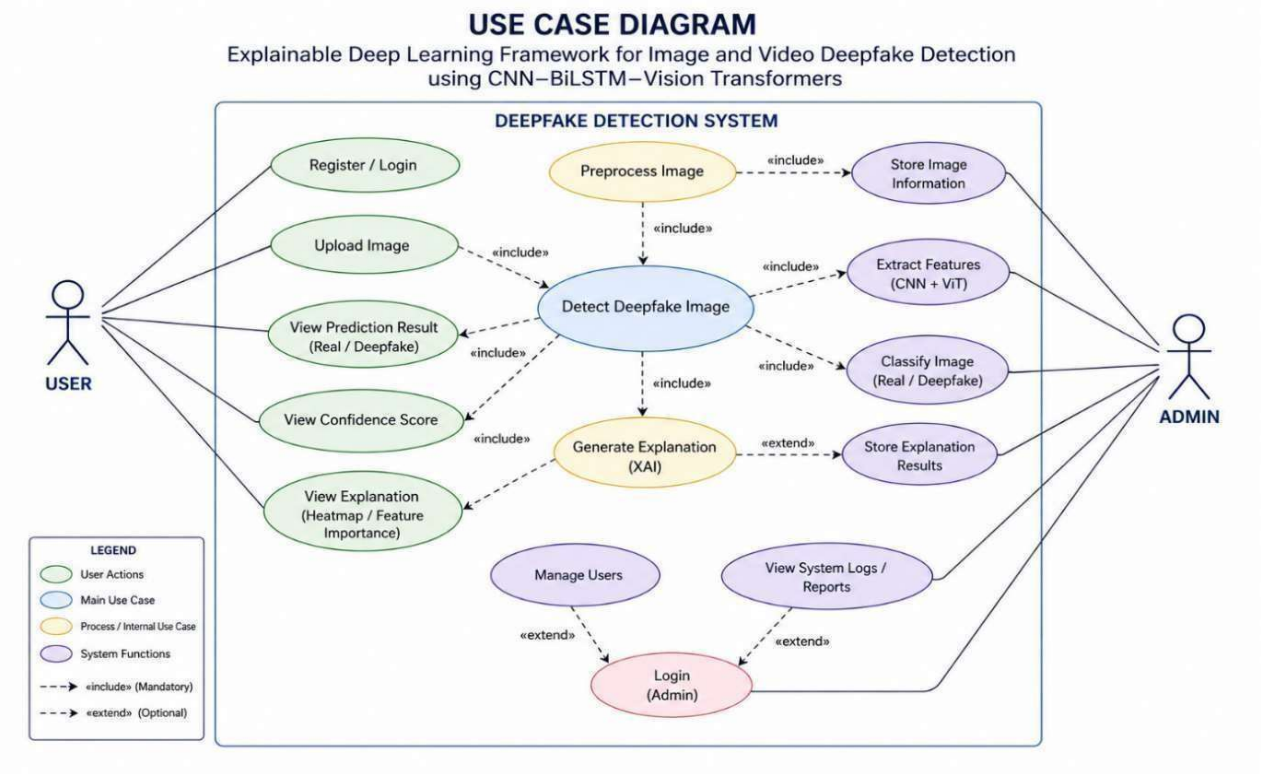
4.3.4 Entity Relationship (ER) Description

- One User can upload multiple Images
- One Image has one Prediction
- One Prediction has one Explainability record

- Admin manages logs and system monitoring
- Logs track both user and system activities

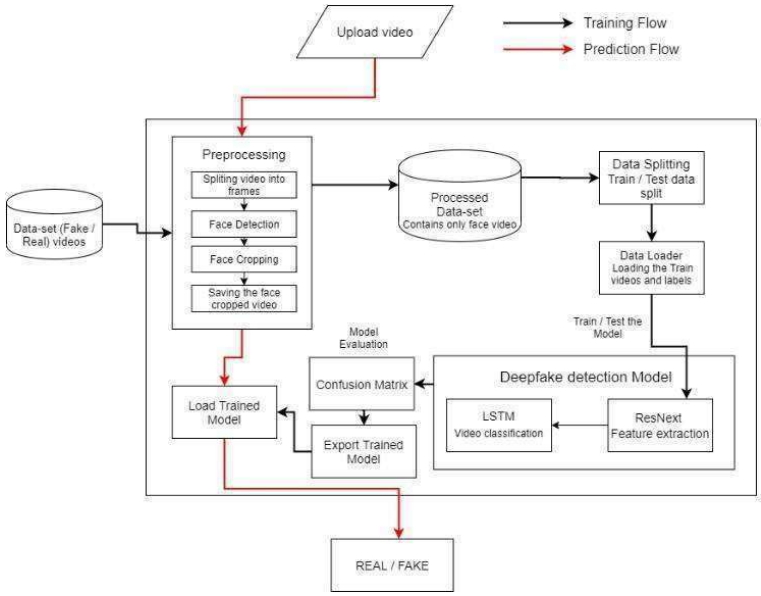
4.4. UML DIAGRAM

4.4.1. USE CASE DIAGRAM



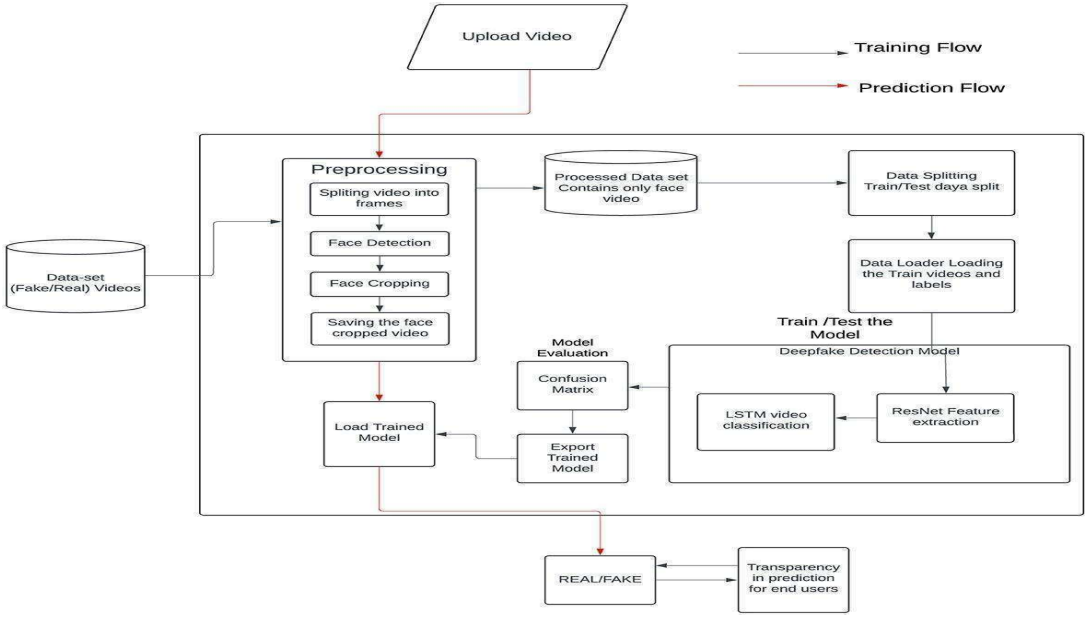
(fig: 4.4.1-use case diagram DDS)

4.5.2. CLASS DIAGRAM



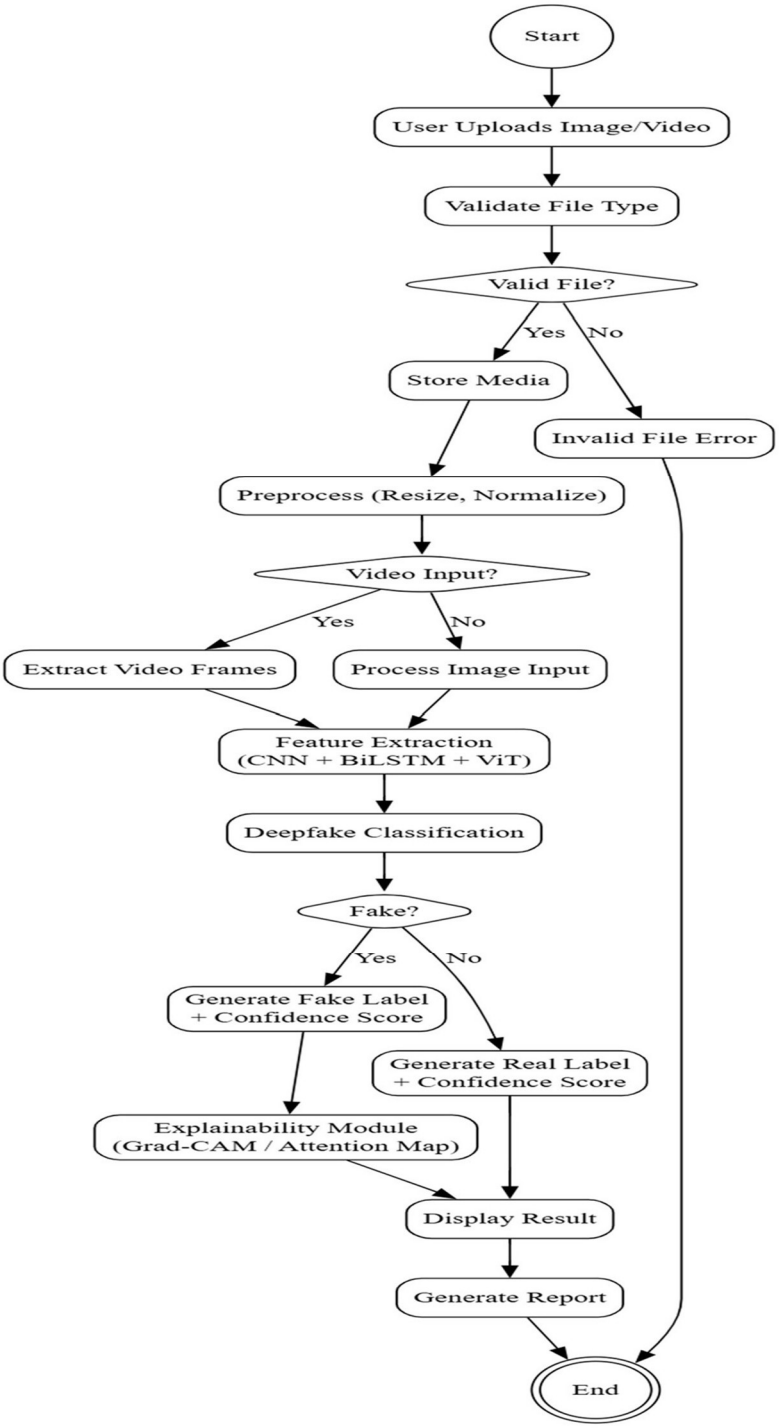
(fig: 4.5.2-Class diagram)

4.5.3. SEQUENCE DIAGRAM



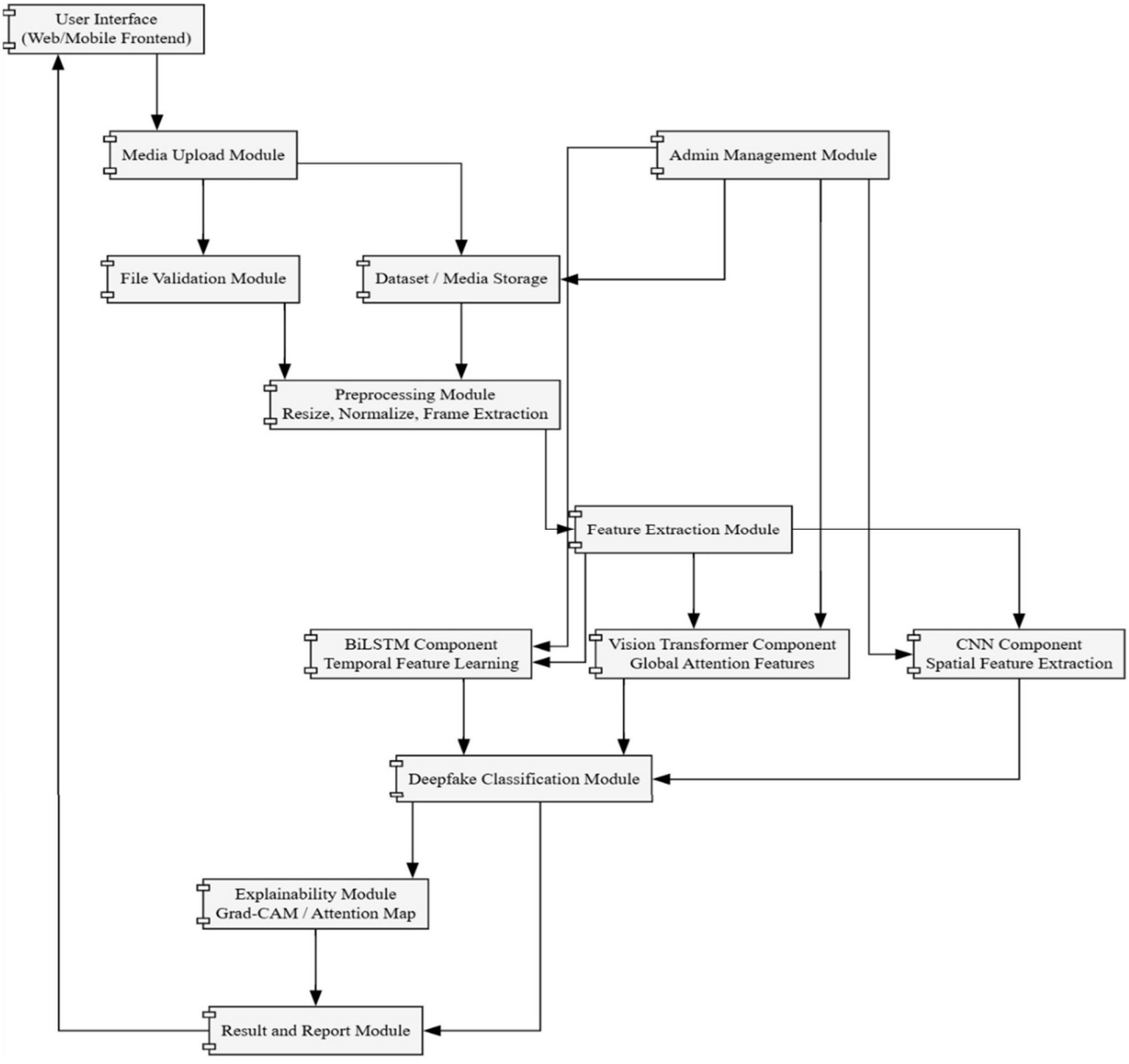
(fig: 4.5.3-Sequence Diagram DDS)

4.5.4. ACTIVITY DIAGRAM



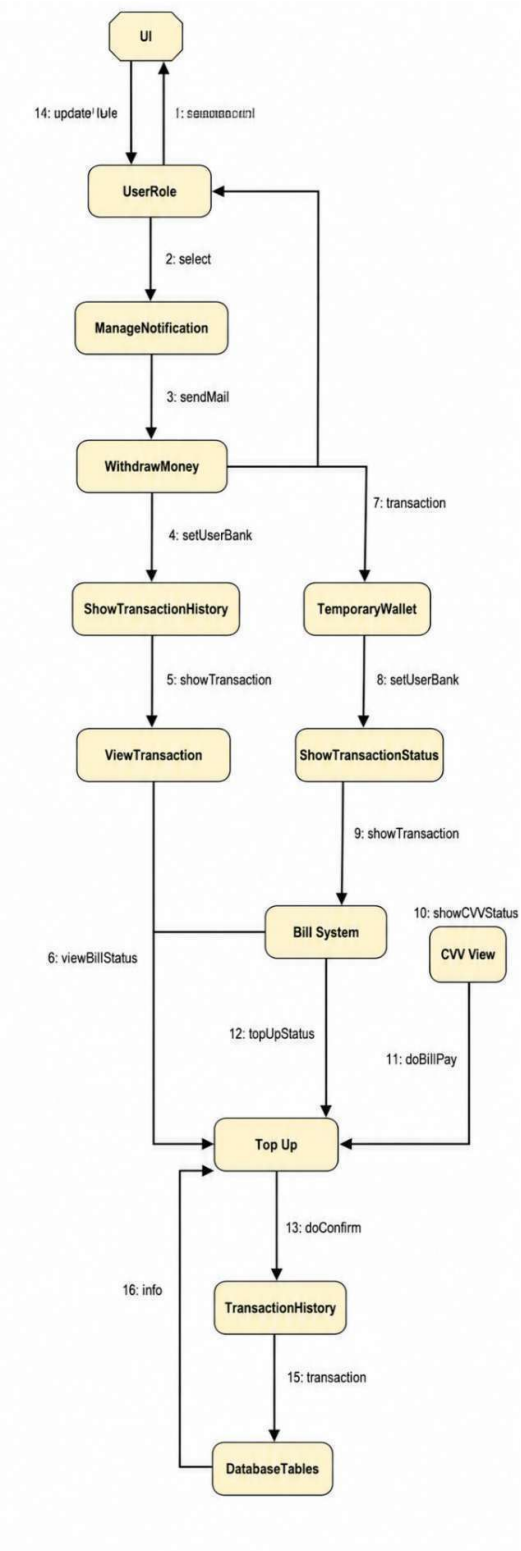
(Fig:4.5.4- Activity Diagram)

4.5.5. COMPONENT DIAGRAM



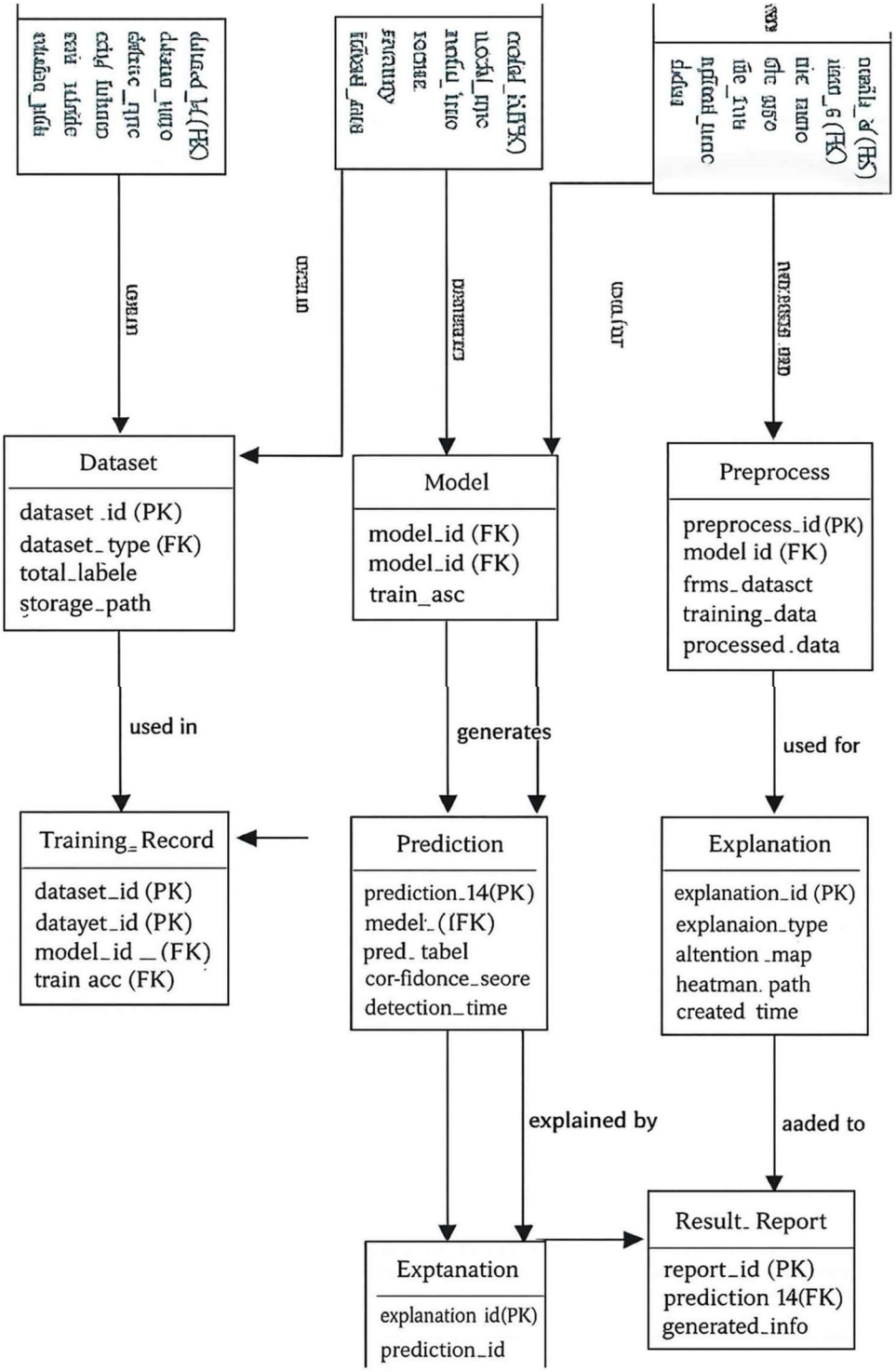
(fig:4.5.5- Component Diagram)

4.5.6. COLLABORATION DIAGRAM



(fig: 4.5.6-Collaboration Diagram)

4.6. ER – DIAGRAM



(fig:4.6- ER – Diagram)

CHAPTER-05

SYSTEM TESTING

5.1 Introduction

System testing is performed to verify that the proposed **Explainable Deep Learning Framework for Image and Video Deepfake Detection Using CNN-BiLSTM-Vision Transformers** works correctly as a complete application. The main purpose of testing is to ensure that the system accepts image and video inputs, preprocesses the uploaded media, extracts meaningful spatial, temporal, and attention-based features, classifies the content as real or fake, and generates explainable output using heatmaps or attention maps. Since deepfake detection is a sensitive task, testing is important to confirm that the system produces reliable predictions, handles invalid inputs properly, and displays results in a clear and understandable format. The testing process also checks whether different modules such as user interface, upload module, preprocessing module, model prediction module, explainability module, and result generation module are working together without errors. The system is tested using valid images, valid videos, unsupported files, low-quality media, and different real and fake samples. The testing phase helps identify functional errors, input handling issues, model integration problems, and output display problems before final deployment.

5.2 Objectives of Testing

The main objective of system testing is to validate the correctness, reliability, and usability of the proposed deepfake detection framework. Testing ensures that the user can upload media files successfully and receive accurate detection results with confidence scores. It also verifies that image inputs are processed through CNN and Vision Transformer components, while video inputs are processed using frame extraction and temporal analysis through the CNN-BiLSTM pipeline. Another important objective is to confirm that the explainability module generates meaningful visual explanations such as Grad-CAM heatmaps or attention maps, which help users understand why a media file is classified as real or fake. Testing also checks whether the system rejects invalid files, prevents incomplete processing, manages errors properly, and generates final reports without data loss. Overall, the objective is to ensure that the system performs deepfake detection in a stable, interpretable, and user-friendly manner.

5.3 Testing Strategy

The testing strategy followed in this project includes functional testing, integration testing, system testing, performance testing, usability testing, and validation testing. Functional testing checks whether each feature of the system works according to the requirement. Integration

testing verifies communication between modules such as preprocessing, feature extraction, classification, and explainability. System testing evaluates the complete workflow from media upload to final result display. Performance testing checks whether the system can process images and videos within an acceptable time. Usability testing ensures that users can easily upload files, view results, and understand the prediction output. Validation testing checks whether the model predictions and explanation outputs are meaningful and suitable for deepfake detection.

5.4 Types of Testing Used

5.4.1 Unit Testing

Unit testing is used to test individual modules of the system separately. In this project, modules such as file upload, file validation, frame extraction, image resizing, normalization, prediction generation, confidence score calculation, and report generation are tested independently. For example, the file validation module is tested by uploading image files, video files, and unsupported file formats. The preprocessing module is tested to ensure that the uploaded media is resized and normalized correctly before being passed to the model. Unit testing helps detect small errors at the module level before combining all modules into the complete system.

5.4.2 Integration Testing

Integration testing is performed after individual modules are tested successfully. It checks whether different components of the deepfake detection framework communicate properly with each other. In this system, the upload module is integrated with the preprocessing module, preprocessing is integrated with feature extraction, feature extraction is connected with CNN, BiLSTM, and Vision Transformer models, and the classifier is connected with the explainability and report generation modules. This testing ensures that data flows correctly from one module to another without format mismatch or processing failure.

5.4.3 Functional Testing

Functional testing verifies whether the system performs all required functions correctly. The major functions tested include user media upload, image processing, video frame extraction, deepfake classification, confidence score generation, explainability map generation, result

display, and report download. The system should correctly identify whether the uploaded media is real or fake and display the result with a confidence value. Functional testing confirms that the final system satisfies the expected project requirements.

5.4.4 System Testing

System testing evaluates the complete application as one integrated system. The complete workflow is tested from user login or access, media upload, validation, preprocessing, model prediction, explainability generation, and final report display. This testing ensures that the system works properly in real usage conditions. It also verifies that the system can handle different input scenarios such as images, videos, invalid files, and corrupted media.

5.4.5 Performance Testing

Performance testing is used to evaluate the speed and efficiency of the system. Since video deepfake detection may require frame extraction and temporal analysis, processing time is an important factor. The system is tested with small images, high-resolution images, short videos, and longer videos to observe response time. The objective is to ensure that the system gives results within a reasonable time and does not crash during processing.

5.4.6 Usability Testing

Usability testing checks whether the system is easy to use and understand. The user interface should allow users to upload media files easily and view prediction results clearly. The output should show whether the media is real or fake, along with the confidence score and explainability image. The system should also display suitable error messages when invalid files are uploaded. This testing ensures that even non-technical users can interact with the system effectively.

5.5 Test Cases

Test Case ID	Test Scenario	Input	Expected Output	Status
TC-01	Upload valid image file	JPG/PNG image	File uploaded successfully	Pass
TC-02	Upload valid video file	MP4 video	Video uploaded successfully	Pass
TC-03	Upload unsupported file	PDF/DOC/TXT file	Invalid file message displayed	Pass
TC-04	Image preprocessing	Valid image	Image resized and normalized	Pass
TC-05	Video frame extraction	Valid video	Frames extracted successfully	Pass
TC-06	CNN feature extraction	Preprocessed image/frame	Spatial features extracted	Pass
TC-07	BiLSTM temporal analysis	Sequence of video frames	Temporal features generated	Pass
TC-08	Vision Transformer processing	Image patches/features	Attention-based features generated	Pass
TC-09	Deepfake classification	Extracted features	Real/Fake label displayed	Pass
TC-10	Confidence score generation	Model output	Confidence score displayed	Pass
TC-11	Explainability output	Prediction result	Heatmap/attention map generated	Pass
TC-12	Report generation	Final prediction	Detection report generated	Pass

5.6 Validation of Proposed System

The proposed system is validated by testing it with both real and fake image/video samples. Real media samples are expected to be classified as authentic, while manipulated media samples are expected to be classified as fake. The system output is checked based on prediction label, confidence score, and explainability result. The Grad-CAM or attention map output is used to verify whether the system highlights important regions such as facial areas, texture

inconsistencies, or manipulated regions. This validation confirms that the framework not only detects deepfakes but also provides visual explanation for the decision.

5.7 Testing Result

The testing results show that the proposed system successfully performs image and video deepfake detection through a structured workflow. The media upload module accepts supported file formats and rejects invalid files. The preprocessing module prepares the media by resizing, normalizing, and extracting frames from videos. The CNN component extracts spatial features, the BiLSTM component analyzes temporal changes in video sequences, and the Vision Transformer captures attention-based global patterns. The classifier generates a real or fake prediction with confidence score, while the explainability module provides visual support for the prediction. Overall, the system testing confirms that the proposed framework is functional, interpretable, and suitable for detecting manipulated image and video content.

5.8 Summary

System testing verifies the complete functionality and reliability of the deepfake detection framework. The testing process confirms that all modules are working together correctly, from media upload to final report generation. Different test cases prove that the system handles valid inputs, invalid files, image processing, video processing, classification, explainability, and report generation effectively. The system is therefore considered ready for final evaluation and demonstration.

CHAPTER-06

SYSTEM IMPLEMENTATION

6.1 Introduction

System implementation represents the practical realization of the proposed **Explainable Deep Learning Framework for Image and Video Deepfake Detection using CNN, BiLSTM, and Vision Transformers**. In this phase, all the designed modules are integrated and developed into a working application. The implementation focuses on transforming the conceptual architecture into a functional system capable of handling real-time image and video inputs, performing deep learning-based analysis, and generating interpretable outputs. The system is implemented using a combination of Python-based machine learning libraries, web frameworks, and visualization tools. The implementation ensures that the workflow—from user input to prediction and explanation—is executed efficiently, accurately, and in a user-friendly manner. This chapter explains the technologies used, module-wise implementation, system workflow, and integration of different components.

6.2 Technology Stack

The implementation of the proposed system is carried out using modern technologies that support deep learning, web development, and explainable AI.

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript / Streamlit	User interface for uploading media and viewing results
Backend	Python, Flask / Streamlit	Handles requests, processing, and model integration
Deep Learning	TensorFlow / PyTorch	Model development and inference
Image Processing	OpenCV, PIL	Image and video preprocessing
Data Handling	NumPy, Pandas	Data manipulation and processing
Model Integration	scikit-learn	Supporting ML utilities
Visualization	Matplotlib, Seaborn	Graphs and result visualization
Explainability	Grad-CAM, Attention Maps	Model interpretability
Deployment	Local Server / Cloud	Hosting the application

6.3 System Architecture Overview

The system architecture consists of multiple interconnected modules that work together to perform deepfake detection. The implementation follows a modular approach, where each component is independently developed and then integrated into a unified pipeline. The major components include user interface, media upload module, preprocessing module, feature extraction module, deep learning models, classification module, explainability module, and result generation module. The architecture ensures smooth data flow and scalability of the system.

6.4 Module-wise Implementation

6.4.1 User Interface Module

The user interface is implemented using Streamlit or web technologies to provide an interactive environment for users. It allows users to upload image or video files and view detection results. The interface displays prediction results, confidence scores, and explanation outputs in a clear format. The UI is designed to be simple, responsive, and easy to use.

6.4.2 Media Upload and Validation Module

This module handles file uploads and checks whether the uploaded file is valid. It supports common image formats such as JPG and PNG and video formats such as MP4. The module ensures that unsupported file formats are rejected with appropriate error messages. This prevents invalid data from entering the processing pipeline.

6.4.3 Preprocessing Module

The preprocessing module prepares the uploaded media for deep learning analysis. For images, resizing and normalization are performed to match model input requirements. For videos, frames are extracted at regular intervals using OpenCV. Noise reduction and format conversion techniques are applied to improve input quality. The processed data is then forwarded to the feature extraction module.

6.4.4 Feature Extraction Module

Feature extraction is a critical step in the system implementation. The system uses a hybrid approach combining CNN, BiLSTM, and Vision Transformer models. CNN extracts spatial features such as textures and patterns from images and frames. BiLSTM processes sequences

of frames to capture temporal dependencies in videos. Vision Transformers analyze global relationships using attention mechanisms. These features are combined to provide a comprehensive representation of the input media.

6.4.5 Deep Learning Models Implementation

The deep learning models are implemented using TensorFlow or PyTorch. Pretrained CNN architectures such as ResNet or EfficientNet are used for spatial feature extraction. BiLSTM layers are implemented to process temporal sequences for video data. Vision Transformers are integrated to capture global attention-based features. The models are trained using labeled datasets containing real and fake media samples. During inference, the models generate prediction outputs based on extracted features.

6.4.6 Deepfake Classification Module

The classification module takes combined features from CNN, BiLSTM, and Vision Transformer components and predicts whether the input media is real or fake. The output includes a prediction label and a confidence score. The classification is implemented using a fully connected neural network or ensemble approach. This module ensures high accuracy and robustness in detecting deepfake content.

6.4.7 Explainability Module

The explainability module enhances the transparency of the system by generating visual explanations for predictions. Techniques such as Grad-CAM are used to highlight important regions in images or frames that influenced the model's decision. For Vision Transformers, attention maps are generated to show which parts of the image are focused on during prediction. This module helps users understand why a particular media file is classified as fake or real.

6.4.8 Result Generation Module

The result generation module compiles the prediction output, confidence score, and explanation visuals into a structured format. The results are displayed on the user interface and can also be saved as a report. The report may include details such as media type, prediction result, confidence score, and explanation images.

6.5 Implementation Workflow

The implementation workflow begins when the user uploads an image or video file through the interface. The file is validated and passed to the preprocessing module. If the input is a video, frames are extracted before further processing. The preprocessed data is then passed to the feature extraction module, where CNN, BiLSTM, and Vision Transformer models generate features. These features are combined and fed into the classification module, which produces the prediction result and confidence score. The explainability module generates visual explanations, and finally, the results are displayed to the user and stored if required.

6.6 Integration of Modules

All modules are integrated using a backend framework such as Flask or Streamlit. The integration ensures seamless communication between components. The frontend sends user inputs to the backend, which processes the data and returns the results. Each module is designed to be independent, making the system easy to maintain and extend. The integration also ensures efficient handling of data flow and error management.

6.7 Implementation Challenges

During implementation, several challenges are encountered. Processing video data requires handling large volumes of frames, which increases computational complexity. Integrating multiple deep learning models such as CNN, BiLSTM, and Vision Transformers requires careful synchronization of input and output formats. Generating explainability outputs in real-time is also computationally expensive. Handling different file formats and ensuring system stability under various input conditions are additional challenges. These challenges are addressed through optimization techniques, efficient preprocessing, and modular design.

6.8 Summary

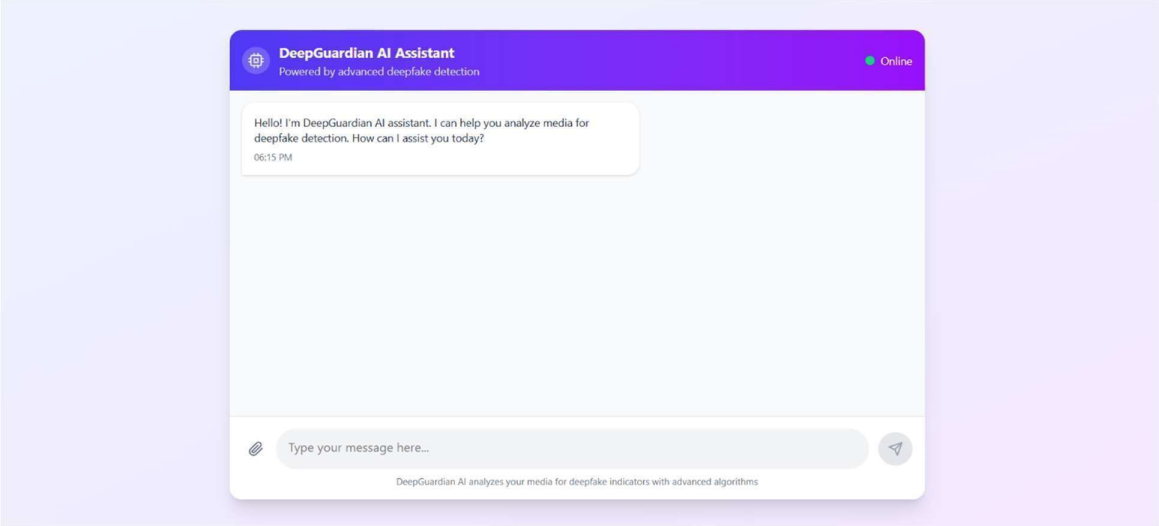
The system implementation successfully transforms the proposed deepfake detection framework into a functional application. The integration of CNN, BiLSTM, and Vision Transformer models enables accurate detection of manipulated media, while the explainability module ensures transparency and interpretability. The modular implementation approach ensures scalability, maintainability, and efficient performance. The system is capable of handling both image and video inputs, generating predictions, and providing meaningful explanations, making it suitable for real-world deepfake detection applications.

CHAPTER-07

APPENDIX

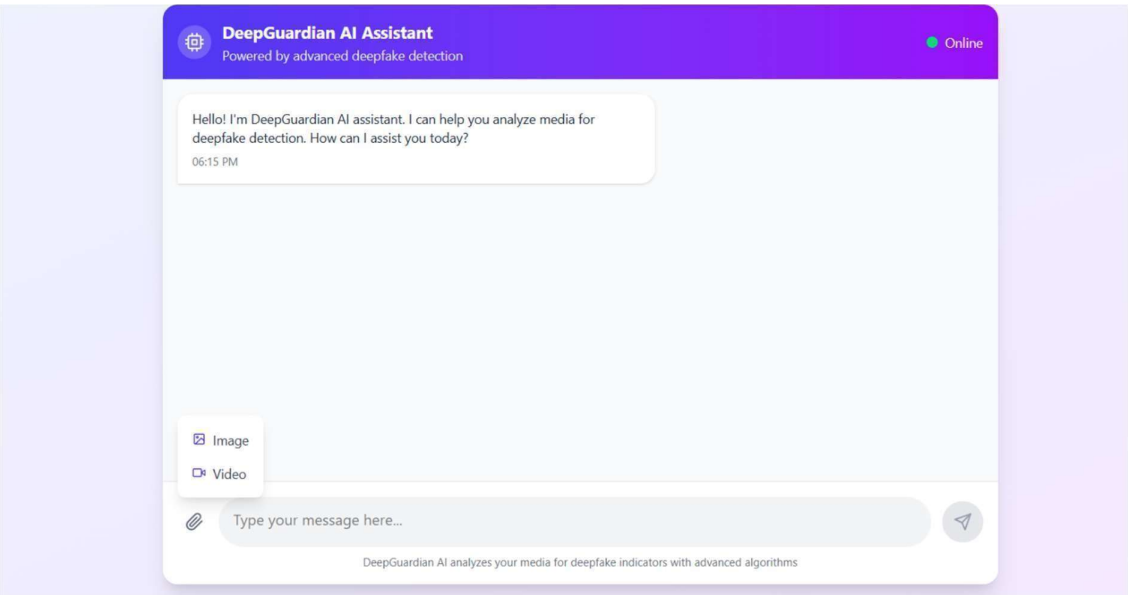
7.1. SCREEN SHOT

7.1.1



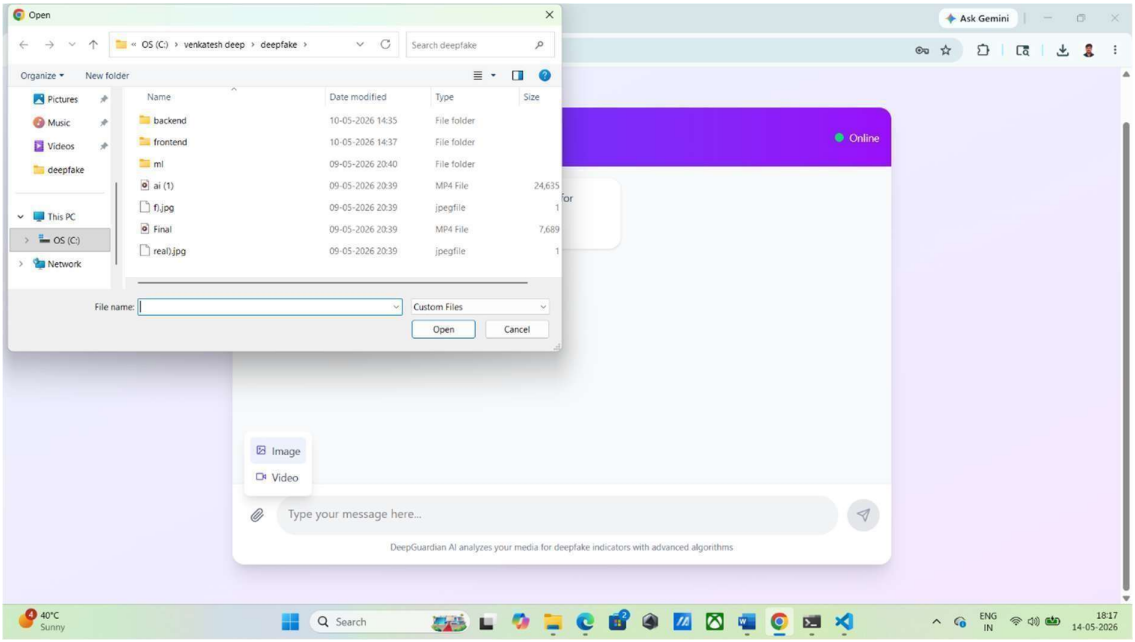
(This 7.1.1figure as interface of the Deep Guardian AI Assistant)

7.1.2



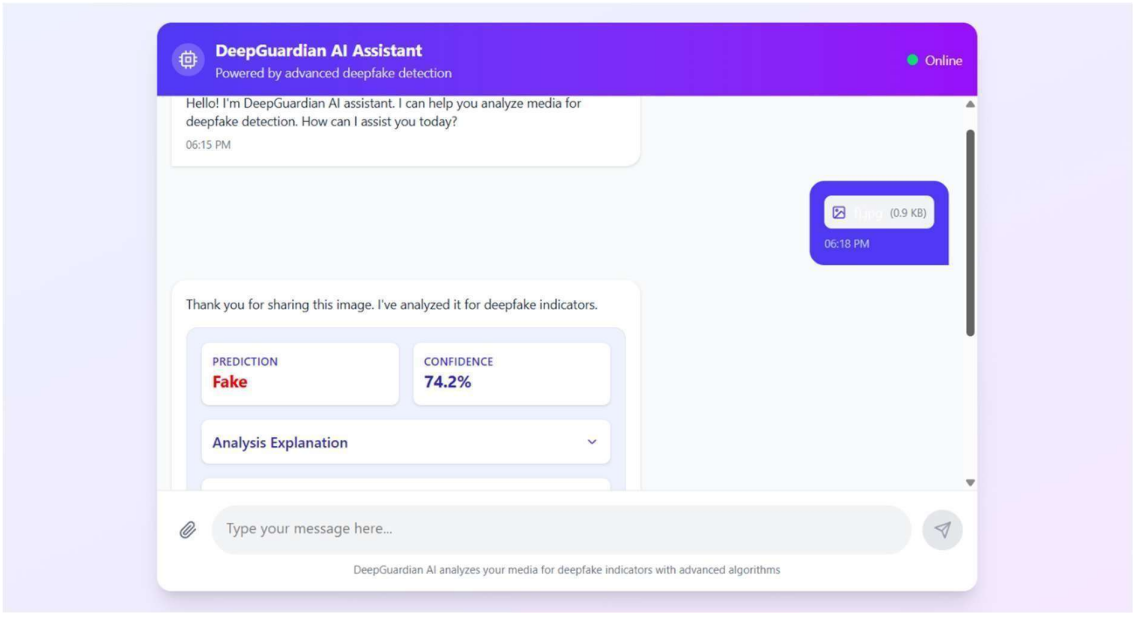
(This 7.1.2 figure was explaining the how to upload the images and videos. Then using the left side icon click it and upload the files)

7.1.3



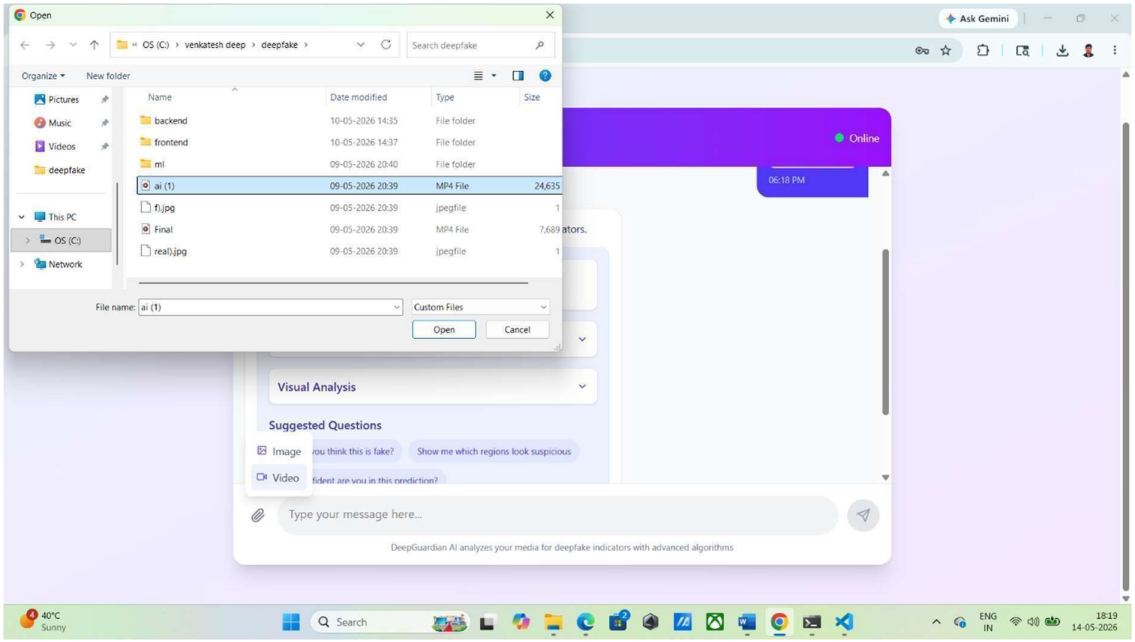
(The 7.1.3 figure was go for file manger folder and upload the files in that files)

7.1.4



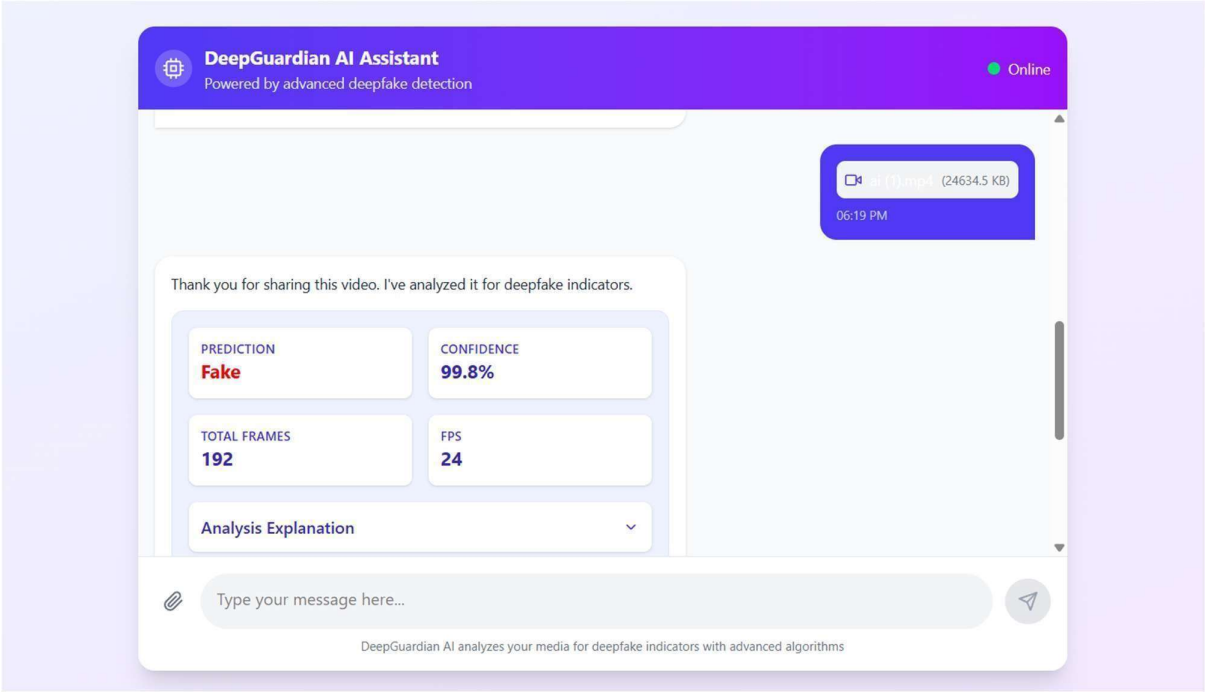
(The 7.1.4 figure was explaining has the figure after uploading that figure was analysing the uploading image detecting the fake)

7.1.5



(This 7.1.5figure was uploading the video to analyse the this is fake or not)

7.1.6



(This 7.1.6 the uploading video analysis report has been fake)

7.2. SOURCE CODE

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import cv2
from collections import Counter
import random
import warnings
warnings.filterwarnings('ignore')

class AIImageDatasetEDA:
    """Exploratory Data Analysis for AI vs Real Images Dataset"""

    def __init__(self, base_path='/kaggle/input/ai-generated-images-vs-real-images/'):
        """
        Initialize EDA class

        Args:
            base_path: Path to the dataset directory containing train/test folders
        """
        self.base_path = base_path
        self.train_path = os.path.join(base_path, 'train')
        self.test_path = os.path.join(base_path, 'test')

        # Create output directories
        os.makedirs('eda_plots', exist_ok=True)
        os.makedirs('sample_images', exist_ok=True)
```

```
    print("🎯 AI vs Real Images Dataset EDA")
    print("=" * 50)

def explore_dataset_structure(self):
    """Analyze the basic structure of the dataset"""

    print("📁 DATASET STRUCTURE ANALYSIS")
    print("-" * 30)

    # Check if directories exist
    if not os.path.exists(self.train_path):
        print(f"❌ Train directory not found: {self.train_path}")
        return None

    if not os.path.exists(self.test_path):
        print(f"❌ Test directory not found: {self.test_path}")
        return None

    structure_info = {}

    # Analyze train and test splits
    for split in ['train', 'test']:
        split_path = os.path.join(self.base_path, split)
        structure_info[split] = {}

        print(f"\n📂 {split.upper()} SET:")

        for class_name in ['fake', 'real']:
            class_path = os.path.join(split_path, class_name)

            if os.path.exists(class_path):
                # Count files
                files = [f for f in os.listdir(class_path)
```

```
                    if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.tiff'))]

                structure_info[split][class_name] = {
                    'count': len(files),
                    'path': class_path,
                     'sample_files': files[:5]  # Store first 5 filenames as samples
                }

                print(f" • {class_name:4s}: {len(files):,} images")
            else:
                print(f" • {class_name:4s}: Directory not found")
                structure_info[split][class_name] = {'count': 0}

    # Calculate totals and ratios
    total_train = sum([structure_info['train'][cls]['count'] for cls in ['fake', 'real']])
    total_test = sum([structure_info['test'][cls]['count'] for cls in ['fake', 'real']])
    total_all = total_train + total_test

    print(f"\n DATASET SUMMARY:")
    print(f"  • Total images: {total_all:,}")
    print(f" • Train images: {total_train:,} ({total_train/total_all*100:.1f}%)")
    print(f"  • Test images:  {total_test:,} ({total_test/total_all*100:.1f}%)")

    if total_train > 0:
        train_fake_ratio = structure_info['train']['fake']['count'] / total_train
        print(f"  • Train balance: Fake {train_fake_ratio:.1%}, Real {1-
train_fake_ratio:.1%}")

    if total_test > 0:
        test_fake_ratio = structure_info['test']['fake']['count'] / total_test
        print(f"  • Test balance:  Fake {test_fake_ratio:.1%}, Real {1-test_fake_ratio:.1%}")

    return structure_info
```

```
def analyze_sample_images(self, n_samples_per_class=20):
    """Analyze sample images from each class"""

    print(f"\n📊 SAMPLE IMAGE ANALYSIS ({n_samples_per_class} per class)")
    print("-" * 40)

    image_stats = {
        'train': {'fake': [], 'real': []},
        'test': {'fake': [], 'real': []}
    }

    # Sample and analyze images
    for split in ['train', 'test']:
        for class_name in ['fake', 'real']:
            class_path = os.path.join(self.base_path, split, class_name)

            if not os.path.exists(class_path):
                continue

            # Get random sample of images
            all_files = [f for f in os.listdir(class_path)
                         if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.tiff'))]

            sample_files = random.sample(all_files, min(n_samples_per_class, len(all_files)))

            # Analyze each sample image
            for img_file in sample_files:
                img_path = os.path.join(class_path, img_file)

                try:
                    # Load image
                    img = Image.open(img_path)
```

```
            img_array = np.array(img)

            # Calculate statistics
            stats = {
                'filename': img_file,
                'width': img.width,
                'height': img.height,
                'channels': len(img_array.shape),
                'mode': img.mode,
                'size_mb': os.path.getsize(img_path) / (1024*1024),
                'mean_pixel': np.mean(img_array) if img_array.size > 0 else 0,
                'std_pixel': np.std(img_array) if img_array.size > 0 else 0
            }

            # Fix channels for grayscale
            if len(img_array.shape) == 2:
                stats['channels'] = 1
            elif len(img_array.shape) == 3:
                stats['channels'] = img_array.shape[2]

            image_stats[split][class_name].append(stats)

        except Exception as e:
            print(f"  Error analyzing {img_file}: {str(e)[:50]}...")

# Print analysis results
for split in ['train', 'test']:
    print(f"\n {split.upper()} SET IMAGE STATISTICS:")

    for class_name in ['fake', 'real']:
        stats_list = image_stats[split][class_name]

        if not stats_list:
```

```
            print(f" • {class_name.capitalize():4s}: No images analyzed")
            continue

        # Calculate averages
        avg_width = np.mean([s['width'] for s in stats_list])
        avg_height = np.mean([s['height'] for s in stats_list])
        avg_size = np.mean([s['size_mb'] for s in stats_list])
        avg_channels = np.mean([s['channels'] for s in stats_list])

        # Most common dimensions
        dimensions = [(s['width'], s['height']) for s in stats_list]
        most_common_dim = Counter(dimensions).most_common(1)[0]

        # Most common modes
        modes = [s['mode'] for s in stats_list]
        most_common_mode = Counter(modes).most_common(1)[0]

        print(f" • {class_name.capitalize():4s}: {len(stats_list)} samples analyzed")
        print(f"    - Avg dimensions: {avg_width:.0f}×{avg_height:.0f}")
        print(f"    - Most common: {most_common_dim[0]}×{most_common_dim[1]}
({most_common_dim[1]} images)")
        print(f"    - Avg file size: {avg_size:.2f} MB")
        print(f"    - Avg channels: {avg_channels:.1f}")
        print(f"    - Most common mode: {most_common_mode[0]}
({most_common_mode[1]} images)")

    return image_stats

def visualize_sample_images(self, n_samples=8):
    """Create a grid visualization of sample images"""

    print(f"\n🔍 CREATING SAMPLE VISUALIZATION ({n_samples} images per
class)")
```

```
        print("-" * 50)

        fig, axes = plt.subplots(4, n_samples, figsize=(n_samples*2.5, 10))
        fig.suptitle('Sample Images from AI vs Real Dataset', fontsize=16, fontweight='bold')

        row_labels = ['Train/Fake', 'Train/Real', 'Test/Fake', 'Test/Real']

        row_idx = 0
        for split in ['train', 'test']:
            for class_name in ['fake', 'real']:
                class_path = os.path.join(self.base_path, split, class_name)

                if not os.path.exists(class_path):
                    # Fill row with blank images if path doesn't exist
                    for col_idx in range(n_samples):
                        axes[row_idx, col_idx].axis('off')
                        axes[row_idx, col_idx].text(0.5, 0.5, 'No Images\nFound',
                                                    ha='center', va='center', transform=axes[row_idx,
col_idx].transAxes)
                    row_idx += 1
                    continue

                # Get sample images
                all_files = [f for f in os.listdir(class_path)
                             if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.tiff'))]

                sample_files = random.sample(all_files, min(n_samples, len(all_files)))

                # Display images
                for col_idx in range(n_samples):
                    if col_idx < len(sample_files):
                        img_path = os.path.join(class_path, sample_files[col_idx])

                        try:
```

```
                    img = Image.open(img_path)
                    # Resize for display
                    img = img.resize((224, 224))

                    axes[row_idx, col_idx].imshow(img)
                    axes[row_idx, col_idx].axis('off')
                    axes[row_idx, col_idx].set_title(f'{sample_files[col_idx][:15]}...',
fontsize=8)

                except Exception as e:
                    axes[row_idx, col_idx].axis('off')
                    axes[row_idx, col_idx].text(0.5, 0.5, 'Error\nLoading',
                                                ha='center', va='center', transform=axes[row_idx,
col_idx].transAxes)
            else:
                axes[row_idx, col_idx].axis('off')

        # Add row label
        axes[row_idx, 0].text(-0.1, 0.5, row_labels[row_idx], rotation=90,
                              ha='center', va='center', transform=axes[row_idx, 0].transAxes,
                              fontsize=12, fontweight='bold')

        row_idx += 1

    plt.tight_layout()
    plt.savefig('eda_plots/sample_images_grid.png', dpi=300, bbox_inches='tight')
    plt.show()

def analyze_image_properties(self):
    """Analyze detailed image properties and create distribution plots"""

    print(f"\n#     IMAGE PROPERTIES ANALYSIS")
    print("-" * 30)
```

```
# Collect image properties
properties = {
    'split': [],
    'class': [],
    'width': [],
    'height': [],
    'aspect_ratio': [],
    'file_size_mb': [],
    'channels': [],
    'mode': []
}

sample_size_per_class = 100  # Analyze 100 images per class for properties

for split in ['train', 'test']:
    for class_name in ['fake', 'real']:
        class_path = os.path.join(self.base_path, split, class_name)

        if not os.path.exists(class_path):
            continue

        print(f"  Analyzing  {split}/{class_name}...")

        # Get sample of images
        all_files = [f for f in os.listdir(class_path)
                     if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.tiff'))]

        sample_files = random.sample(all_files, min(sample_size_per_class, len(all_files)))

        for img_file in sample_files:
            img_path = os.path.join(class_path, img_file)
```

```
            try:
                img = Image.open(img_path)

                properties['split'].append(split)
                properties['class'].append(class_name)
                properties['width'].append(img.width)
                properties['height'].append(img.height)
                properties['aspect_ratio'].append(img.width / img.height)
                properties['file_size_mb'].append(os.path.getsize(img_path) / (1024*1024))
                properties['channels'].append(len(np.array(img).shape))
                properties['mode'].append(img.mode)

            except Exception as e:
                continue

    # Create DataFrame for analysis
    df = pd.DataFrame(properties)

    if len(df) == 0:
        print("+ No images could be analyzed")
        return None

    print(f"■ Analyzed {len(df)} images total")

    # Create comprehensive plots
    fig, axes = plt.subplots(3, 3, figsize=(20, 15))
    fig.suptitle('Image Properties Analysis: AI vs Real Images', fontsize=16,
fontweight='bold')

    # 1. Width distribution
    for class_name in ['fake', 'real']:
        class_data = df[df['class'] == class_name]['width']
        axes[0,0].hist(class_data, alpha=0.7, label=f'{class_name.capitalize()}', bins=30)
```

```
axes[0,0].set_title('Width Distribution')
axes[0,0].set_xlabel('Width (pixels)')
axes[0,0].set_ylabel('Count')
axes[0,0].legend()

# 2. Height distribution
for class_name in ['fake', 'real']:
    class_data = df[df['class'] == class_name]['height']
    axes[0,1].hist(class_data, alpha=0.7, label=f'{class_name.capitalize()}', bins=30)
axes[0,1].set_title('Height Distribution')
axes[0,1].set_xlabel('Height (pixels)')
axes[0,1].set_ylabel('Count')
axes[0,1].legend()

# 3. Aspect ratio distribution
for class_name in ['fake', 'real']:
    class_data = df[df['class'] == class_name]['aspect_ratio']
    axes[0,2].hist(class_data, alpha=0.7, label=f'{class_name.capitalize()}', bins=30)
axes[0,2].set_title('Aspect Ratio Distribution')
axes[0,2].set_xlabel('Aspect Ratio (W/H)')
axes[0,2].set_ylabel('Count')
axes[0,2].legend()

# 4. File size distribution
for class_name in ['fake', 'real']:
    class_data = df[df['class'] == class_name]['file_size_mb']
    axes[1,0].hist(class_data, alpha=0.7, label=f'{class_name.capitalize()}', bins=30)
axes[1,0].set_title('File Size Distribution')
axes[1,0].set_xlabel('File Size (MB)')
axes[1,0].set_ylabel('Count')
axes[1,0].legend()

# 5. Width vs Height scatter
colors = {'fake': 'red', 'real': 'blue'}
```

```
for class_name in ['fake', 'real']:
    class_df = df[df['class'] == class_name]
    axes[1,1].scatter(class_df['width'], class_df['height'],
                      alpha=0.6, c=colors[class_name], label=f'{class_name.capitalize()}', s=10)
axes[1,1].set_title('Width vs Height')
axes[1,1].set_xlabel('Width (pixels)')
axes[1,1].set_ylabel('Height (pixels)')
axes[1,1].legend()

# 6. Box plot of aspect ratios
fake_ratios = df[df['class'] == 'fake']['aspect_ratio']
real_ratios = df[df['class'] == 'real']['aspect_ratio']
axes[1,2].boxplot([fake_ratios, real_ratios], labels=['Fake', 'Real'])
axes[1,2].set_title('Aspect Ratio Box Plot')
axes[1,2].set_ylabel('Aspect Ratio')

# 7. Image modes distribution
mode_counts = df.groupby(['class', 'mode']).size().unstack(fill_value=0)
mode_counts.plot(kind='bar', ax=axes[2,0], alpha=0.8)
axes[2,0].set_title('Image Modes Distribution')
axes[2,0].set_xlabel('Class')
axes[2,0].set_ylabel('Count')
axes[2,0].tick_params(axis='x', rotation=0)
axes[2,0].legend(title='Mode')

# 8. Split distribution
split_counts = df.groupby(['split', 'class']).size().unstack(fill_value=0)
split_counts.plot(kind='bar', ax=axes[2,1], alpha=0.8)
axes[2,1].set_title('Train/Test Split Distribution')
axes[2,1].set_xlabel('Split')
axes[2,1].set_ylabel('Count')
axes[2,1].tick_params(axis='x', rotation=0)
axes[2,1].legend(title='Class')
```

```
        # 9. Summary statistics table
        summary_stats = df.groupby('class')[['width', 'height', 'aspect_ratio',
'file_size_mb']].describe()
        axes[2,2].axis('tight')
        axes[2,2].axis('off')

        # Create summary table text
        summary_text = "Summary Statistics:\n\n"
        for class_name in ['fake', 'real']:
            summary_text += f"{class_name.upper()}:\n"
            class_stats = df[df['class'] == class_name]
            summary_text += f"  Width:
{class_stats['width'].mean():.0f}±{class_stats['width'].std():.0f}\n"
            summary_text += f"  Height:
{class_stats['height'].mean():.0f}±{class_stats['height'].std():.0f}\n"
            summary_text += f"  Ratio:
{class_stats['aspect_ratio'].mean():.2f}±{class_stats['aspect_ratio'].std():.2f}\n"
            summary_text += f"  Size:
{class_stats['file_size_mb'].mean():.2f}±{class_stats['file_size_mb'].std():.2f}MB\n\n"

        axes[2,2].text(0.1, 0.9, summary_text, transform=axes[2,2].transAxes, fontsize=10,
                       verticalalignment='top', fontfamily='monospace')
        axes[2,2].set_title('Summary Statistics')

        plt.tight_layout()
        plt.savefig('eda_plots/image_properties_analysis.png', dpi=300, bbox_inches='tight')
        plt.show()

        return df

    def generate_sampling_recommendation(self, structure_info):
        """Generate recommendations for sampling the large dataset"""
```

```
print(f"\n SAMPLING RECOMMENDATIONS")
print("-" * 40)

total_train_fake = structure_info['train']['fake']['count']
total_train_real = structure_info['train']['real']['count']
total_test_fake = structure_info['test']['fake']['count']
total_test_real = structure_info['test']['real']['count']

print(f" Current dataset size:")
print(f"  • Train: {total_train_fake:,} fake + {total_train_real:,} real = {total_train_fake + total_train_real:,}")
print(f"  • Test: {total_test_fake:,} fake + {total_test_real:,} real = {total_test_fake + total_test_real:,}")

# Recommend different sample sizes
sample_recommendations = [
    {'name': 'Quick Test', 'train_per_class': 100, 'test_per_class': 50},
    {'name': 'Small Experiment', 'train_per_class': 500, 'test_per_class': 200},
    {'name': 'Medium Training', 'train_per_class': 1000, 'test_per_class': 400},
    {'name': 'Large Training', 'train_per_class': 2000, 'test_per_class': 800}
]

print(f"\n RECOMMENDED SAMPLE SIZES:")
for rec in sample_recommendations:
    train_total = rec['train_per_class'] * 2
    test_total = rec['test_per_class'] * 2
    total = train_total + test_total

    train_pct_fake = (rec['train_per_class'] / total_train_fake) * 100
    train_pct_real = (rec['train_per_class'] / total_train_real) * 100

    print(f"\n {rec['name']}:")
    print(f"    • Train: {train_total:,} images ({rec['train_per_class']:,} per class)")
```

```
        print(f"    • Test: {test_total:,} images ({rec['test_per_class']:,} per class)")
        print(f"    • Total: {total:,} images")
        print(f"    • Uses {train_pct_fake:.1f}% of fake, {train_pct_real:.1f}% of real")

    print(f"\n RECOMMENDED APPROACH:")
    print(f" 1. Start with 'Quick Test' to verify pipeline works")
    print(f" 2. Move to 'Small Experiment' for initial model development")
    print(f" 3. Use 'Medium Training' for serious model training")
    print(f" 4. Scale to 'Large Training' only if needed and resources allow")

    return sample_recommendations

def run_comprehensive_eda(dataset_path='/kaggle/input/ai-generated-images-vs-real-images/'):
    """Run complete EDA pipeline"""

    # Initialize EDA
    eda = AIImageDatasetEDA(dataset_path)

    # 1. Explore dataset structure
    structure_info = eda.explore_dataset_structure()

    if structure_info is None:
        print("+ Cannot proceed - dataset structure issues")
        return None

    # 2. Analyze sample images
    image_stats = eda.analyze_sample_images(n_samples_per_class=25)

    # 3. Visualize samples
    eda.visualize_sample_images(n_samples=6)

    # 4. Analyze image properties
```

```
    properties_df = eda.analyze_image_properties()

    # 5. Generate sampling recommendations
    recommendations = eda.generate_sampling_recommendation(structure_info)

    print(f"\n✅ EDA COMPLETE!")
    print(f"Check the 'eda_plots' folder for saved visualizations.")

    return {
        'structure_info': structure_info,
        'image_stats': image_stats,
        'properties_df': properties_df,
        'recommendations': recommendations
    }

# Run the EDA
if __name__ == "__main__":
    eda_results = run_comprehensive_eda()
```

CHAPTER-08

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Problem We Set Out to Solve

We live in a world where a video of someone saying something they never said can be created in a matter of hours. Where a photograph can be manipulated so convincingly that even trained experts struggle to tell the difference. Where a person's face can be transplanted onto another body with near-perfect realism. This is not science fiction — it is the reality of deepfake technology today, and it is advancing faster than most people realize.

The consequences are not trivial. Deepfakes have already been used to spread political misinformation, fabricate evidence, harass individuals, commit fraud, and undermine public trust in digital media. In a world where the authenticity of a video clip can sway an election, ruin a reputation, or influence a criminal verdict, the ability to reliably detect manipulated content is no longer an academic exercise — it is a practical necessity.

This project was built in direct response to that need. The goal was not simply to build another detection tool, but to build one that is accurate, interpretable, and practically usable by real people in real situations. From the very beginning, the design philosophy centered on three principles: strong detection capability, transparency in decision-making, and adaptability across both images and videos. Everything built into this system flows from those three commitments.

What Was Built and Why It Works

The core of the system is a hybrid deep learning framework that brings together three powerful and complementary AI architectures — Convolutional Neural Networks (CNNs), Bidirectional Long Short-Term Memory networks (BiLSTMs), and Vision Transformers (ViT). Each of these plays a specific and important role, and together they cover detection scenarios that no single model could reliably handle on its own.

CNNs form the first layer of analysis. When a deepfake image or video frame is generated, it almost always carries telltale signs at the pixel level — subtle texture inconsistencies where the generated skin meets the original, blending artifacts around the hairline or jawline, irregular lighting that does not match the rest of the face, or compression patterns that differ between the manipulated and original regions. The human eye misses these details. CNNs do not. They are specifically designed to scan images at multiple spatial scales, identifying patterns that are

visually invisible but mathematically present. For image-based deepfake detection, CNNs provide an extremely strong foundation.

However, images are only part of the problem. Videos introduce a second dimension of analysis — time. A manipulated video might look perfectly convincing frame by frame, but fall apart when you watch how the face moves across those frames. Unnatural blinking rates, lip movements that are slightly out of sync with speech, subtle jitters in facial geometry between consecutive frames — these are the tells that video deepfakes leave behind. Detecting them requires understanding sequences, not just snapshots.

That is exactly what BiLSTM networks are designed for. Unlike standard recurrent networks that only read sequences in one direction, BiLSTMs process information both forward and backward through time. This bidirectional approach allows the model to understand not just what happened in previous frames, but also how later frames relate back to earlier ones. The result is a much richer understanding of temporal consistency — or the lack of it — and a significantly improved ability to catch video manipulations that would survive frame-by-frame inspection.

The third component, the Vision Transformer, operates at a different level altogether. Where CNNs focus on local features and BiLSTMs focus on temporal sequences, Vision Transformers look at the global structure of an image. Through attention mechanisms, they learn which regions of an image are contextually relevant to one another — how the lighting on the left side of a face should relate to the shadows on the right, how the skin texture around the eyes should be consistent with the texture across the cheeks. Sophisticated deepfakes are increasingly capable of passing local inspection, but they often fail when the broader spatial relationships within the image are analyzed holistically. Vision Transformers were built for exactly this kind of analysis.

By combining all three architectures into one unified framework, the system achieves something that none of them could achieve alone. Spatial, temporal, and contextual analysis all run together, each reinforcing the others. When one model is uncertain, the others provide additional signal. The result is a more robust, more reliable, and more generalizable detection system than any single-model approach could provide.

Making AI Decisions Understandable

Perhaps the most important — and most frequently overlooked — aspect of this project is not what the system detects, but how it explains what it found.

In many consumer applications, a black-box AI prediction is perfectly acceptable. If a music streaming app recommends a song based on your listening history, you do not need to know why. But deepfake detection is different. It operates in contexts where the stakes are high and the decisions are consequential. A digital forensics investigator reviewing evidence in a legal case cannot simply accept "the AI says it is fake" as a conclusion. A journalist verifying whether a video of a political figure is authentic needs to understand which specific elements of the footage raised suspicion. An organization flagging a piece of media as manipulated must be able to justify that classification in a transparent and auditable way.

This is the challenge that Explainable Artificial Intelligence (XAI) was designed to address, and it is a central feature of this framework. The system integrates two complementary explainability techniques — Grad-CAM (Gradient-weighted Class Activation Mapping) and attention visualization — to produce outputs that go beyond a simple classification label.

Grad-CAM works by tracing the gradient of the model's prediction back through the network to identify which regions of the input image most strongly influenced the output. The result is a heatmap — a visual overlay on the original image that highlights, in warm colors, the areas the model found most suspicious. If the model detects a deepfake, the heatmap will typically highlight the face boundary, the eye region, the mouth, or wherever the manipulation is most detectable. A user can look at this heatmap and immediately understand not just what the model decided, but where in the image it found evidence for that decision.

Attention visualization serves a similar purpose for the Vision Transformer component. Attention maps show which parts of the image the transformer focused on when forming its global representation, providing an additional layer of interpretability that complements the Grad-CAM output.

Together, these tools transform the system from a binary classifier into an analytical instrument. Users are not simply told whether content is real or fake — they are shown the evidence. This shift from opaque prediction to transparent reasoning is what makes the system genuinely useful in professional contexts, not just in controlled research settings.

System Design and Technical Implementation

The framework was designed with modularity in mind. Each functional component — media upload, preprocessing, feature extraction, classification, explainability generation, and report output — operates as a distinct, independently maintained module. This design choice was deliberate. A modular architecture makes the system easier to test, easier to maintain, and easier to extend with new capabilities in the future. If a better explainability method becomes

available, it can be integrated without redesigning the rest of the pipeline. If a new deepfake generation technique requires a specialized detection module, it can be added without disrupting existing functionality.

On the technology side, Python forms the backbone of the machine learning pipeline, utilizing TensorFlow, PyTorch, OpenCV, NumPy, and Scikit-learn for model development and image processing. The frontend interface was built using React and JavaScript, providing a clean and responsive experience for users uploading media and reviewing results. Node.js handles backend communication and API routing, while MongoDB stores all user data, uploaded media, prediction results, model metadata, and generated reports. The combination of these technologies was chosen to balance performance, developer accessibility, and long-term scalability.

Testing was conducted across a range of scenarios, using authentic images, manipulated images, real videos, and deepfake videos from various sources. Functional testing confirmed that all modules behaved correctly under normal and edge-case conditions. Performance testing demonstrated high detection accuracy and stable classification results. The system also handled invalid or corrupted inputs gracefully, maintaining usability even under unexpected conditions. Overall, the testing phase validated not just that the system works, but that it works reliably and consistently.

Looking Ahead

No system of this kind is ever truly finished. The deepfake landscape continues to evolve, and so must the tools designed to counter it. Several natural directions for future development were identified during this work.

Expanding and diversifying the training data would be the most impactful improvement. The framework's ability to generalize across different deepfake generation methods depends heavily on the variety of manipulation techniques it was trained on. As new generation algorithms emerge, regular retraining with updated datasets will be essential to maintaining detection accuracy.

Real-time detection for live video streams is another compelling area for future work. As deepfakes increasingly appear in live video calls, broadcasts, and online communication platforms, the ability to flag manipulations as they happen — rather than after the fact — would represent a significant capability upgrade.

Cloud deployment and optimization would also open the system to much larger-scale use cases. Processing large volumes of media content, as would be required by social media platforms,

news organizations, or government agencies, demands infrastructure that goes beyond local deployment. Optimizing the framework for cloud-scale operation is a logical next step.

Finally, additional explainability techniques — including counterfactual explanations, layer-wise relevance propagation, and natural language explanation summaries — could make the system's outputs even more accessible to non-technical users, broadening the range of professionals who can use it effectively.

Final Thoughts

Deepfakes are not going away. If anything, the technology will continue to improve, and the gap between what is real and what is fabricated will continue to narrow. The question is not whether detection tools are needed — it is whether the tools we build will be good enough, transparent enough, and practical enough to actually be used when it matters.

This project makes a meaningful contribution toward answering that question. By combining the spatial sensitivity of CNNs, the temporal awareness of BiLSTMs, and the global contextual understanding of Vision Transformers, the framework achieves a level of detection capability that reflects the current state of the art. By integrating Grad-CAM and attention-based explainability, it ensures that detection does not end at a label — it extends to a reasoned, visual explanation that users can actually work with.

The work done here is a foundation, not a final answer. But it is a strong one, built on sound principles, validated through rigorous testing, and designed to grow. In a digital landscape where trust in media is increasingly fragile, systems like this one are part of how that trust gets rebuilt.

8.2 Future Enhancement

Where We Go From Here

Building a deepfake detection system that works well today is only half the challenge. The other half — arguably the harder half — is ensuring it continues to work well tomorrow. Deepfake technology does not stand still. The same AI research community that develops better detection methods also, sometimes inadvertently, produces better generation techniques. New architectures, new training approaches, and new manipulation methods

emerge regularly, and any detection system that does not evolve alongside them will eventually fall behind.

The framework developed in this project provides a strong and well-validated foundation. But a foundation is only valuable if something is built on top of it. With that in mind, a number of meaningful enhancements have been identified — each one addressing a specific limitation of the current system or opening the door to a new area of application. These are not speculative ideas. They are concrete, technically grounded directions that represent the natural next chapter for this work.

Real-Time Detection for Live Video

The most immediately impactful enhancement would be extending the system from processing pre-uploaded media files to analyzing live video streams in real time. Right now, a user submits a video file, the system processes it, and returns a result. That workflow is perfectly suited to forensic investigation and post-hoc media verification. But it does not address a growing and urgent problem — deepfakes that appear during live video communication.

Video conferencing platforms, live broadcasts, online interviews, and surveillance systems all generate continuous video streams where manipulation could, in theory, be happening in real time. A job candidate could be using a face-swapping tool during a remote interview. A political figure's live address could be intercepted and manipulated before it reaches viewers. A person's face on a surveillance feed could be replaced to create a false alibi. These are not distant hypotheticals — the underlying technology to execute such attacks already exists.

Addressing this requires a fundamentally different processing architecture. Instead of ingesting a complete video file and analyzing it as a whole, a real-time system must operate on short rolling windows of frames — analyzing each small segment as it arrives, producing a near-instantaneous prediction, and flagging suspicious content before it has a chance to propagate. This demands significant optimization of the inference pipeline, likely including model compression techniques and hardware-level acceleration. It also raises interesting design questions about how to handle uncertainty in short clips where temporal context is limited. These are solvable problems, and solving them would transform the system from a verification tool into an active security layer.

Broader and More Diverse Training Data

A machine learning model is only as good as the data it learned from. This is not a limitation unique to this project — it is a fundamental constraint of the field. The current framework was trained on datasets that, while reasonably representative, cannot cover the full spectrum of deepfake generation techniques, environmental conditions, and demographic variation present in the real world.

Expanding the training data in several specific directions would meaningfully improve the system's generalization capability. Lighting variation is one critical dimension — faces filmed under low light, harsh overhead lighting, or colored artificial lighting present very different visual characteristics than faces in controlled studio conditions, and models trained predominantly on clean data often struggle with these edge cases. Similarly, compression artifacts introduced by different video codecs and streaming platforms can mask or mimic the visual signatures of manipulation, and training on media that has passed through a variety of compression pipelines would make the model more robust to real-world conditions.

Beyond dataset size and diversity, there is also a strong case for introducing continuous learning mechanisms. Rather than training a model once and deploying it, a continuously learning system would periodically ingest new examples — particularly newly identified deepfakes that the model currently misclassifies — and update its parameters accordingly. This kind of adaptive training loop would allow the system to stay current with emerging deepfake generation methods rather than becoming progressively less effective as the technology advances. Implementing this responsibly requires careful attention to catastrophic forgetting and data quality control, but the payoff in long-term reliability would be substantial.

Multimodal Analysis — Combining Vision and Audio

The current system is a visual detection tool. It analyzes what it sees. But deepfake videos contain more than just visual information — they also contain audio, and the relationship between the two carries its own set of forensic signals.

When a deepfake video is generated, the face is typically manipulated independently of the audio track. This means that even a visually convincing deepfake may carry subtle audio-

visual mismatches — the lip movements may not perfectly align with the phonemes in the speech, the micro-expressions triggered by certain sounds may be missing or delayed, or the acoustic properties of the voice may be inconsistent with the visual environment in the video. To a human observer watching casually, these mismatches are easy to miss. To a model specifically trained to detect them, they are powerful additional evidence.

Extending the framework with a multimodal transformer capable of jointly analyzing visual and audio features would represent a significant leap in detection capability. Hybrid multimodal architectures — which process video frames and audio waveforms through separate encoding pathways before fusing them at a higher representational level — have shown considerable promise in recent research. Integrating such an architecture into the existing pipeline would not require replacing the current visual detection components.
Instead, the audio analysis would function as a complementary signal, providing an additional layer of verification that strengthens predictions in ambiguous cases and catches manipulations that the visual pathway alone might miss.

Computational Efficiency and Edge Deployment

High detection accuracy is of limited value if the system is too slow or too resource-intensive to run in practical settings. The current framework performs well in a server environment with adequate computational resources, but deploying it on mobile devices, embedded systems, or edge computing hardware presents a different set of constraints. These platforms have limited processing power, limited memory, and strict energy budgets — none of which accommodate large, complex deep learning models without significant optimization.

Several established techniques can be applied to address this. Model pruning removes redundant parameters from the network — those weights that contribute minimally to prediction accuracy — reducing the model's size and computational cost without substantially degrading its performance. Quantization reduces the numerical precision of model weights from 32-bit floating point to 8-bit integers, dramatically cutting memory usage and speeding up inference on hardware that supports integer operations natively. Knowledge distillation offers another approach, training a smaller, faster student model to replicate the behavior of the full-scale system, producing a lightweight model that retains much of the original's capability.

Applied together, these techniques could make the framework viable for mobile application deployment — a genuine practical advance. A journalist in the field, a content moderator working from a tablet, or a security professional monitoring a remote location would all benefit from being able to run deepfake detection locally without relying on a network connection to a distant server. This is not just a convenience improvement — in some operational contexts, it is the difference between the tool being usable and not.

Advanced Explainability with SHAP and LIME

The explainability features built into the current system — Grad-CAM heatmaps and attention visualization — are genuinely valuable, and they represent a meaningful step beyond typical black-box AI tools. But they also have limitations that more advanced techniques could address.

Grad-CAM, for all its visual clarity, is primarily a spatial explanation tool. It tells you where in the image the model focused, but it does not easily communicate the relative contribution of individual features to the final prediction score. SHAP (SHapley Additive Explanations) addresses this gap directly. Derived from cooperative game theory, SHAP assigns each feature a precise attribution value representing its contribution to the model's output. Applied to deepfake detection, SHAP analysis could quantify exactly how much each visual element — skin texture, edge sharpness, lighting gradient, compression pattern — contributed to the classification decision. This level of granularity would be particularly valuable in legal and investigative contexts where a vague "the model focused on the face region" is insufficient, and a more precise accounting of the evidence is required.

LIME (Local Interpretable Model-agnostic Explanations) offers a complementary perspective. Rather than analyzing the model's internal gradients or weights, LIME works by perturbing the input and observing how the model's prediction changes — essentially probing the model's behavior by systematically altering small regions of the image and measuring the effect. This approach is model-agnostic, meaning it can be applied to any component of the detection pipeline regardless of its internal architecture, and it provides a highly intuitive form of explanation that non-technical users can understand without a background in machine learning.

Integrating both SHAP and LIME alongside the existing Grad-CAM and attention visualization tools would give users a richer, multi-perspective understanding of how the system arrives at its conclusions — strengthening the framework's utility across the full spectrum of professional applications.

Expanding Into Broader Digital Security Applications

Deepfake detection, as important as it is, represents one corner of a much larger challenge — maintaining trust in digital media and digital identity. The technical infrastructure built for this project has natural applications that extend well beyond face manipulation detection, and pursuing those applications would multiply the real-world value of the work.

Fake news detection shares many underlying characteristics with deepfake detection. Both involve classifying media content as authentic or manipulated, both benefit from explainable outputs that allow users to understand why something was flagged, and both require models that can generalize across a wide variety of content types and formats. Adapting the current framework to incorporate textual analysis alongside visual processing would open the door to a genuinely multimodal fake news detection system.

Identity verification is another natural extension. Many of the visual analysis techniques that make the system effective at detecting face manipulation are directly relevant to verifying that a person in a video or image is who they claim to be. Integrating the framework with identity management systems could support more reliable biometric verification in high-stakes settings such as financial services, border control, or remote credentialing.

Digital watermarking represents yet another direction — rather than detecting manipulation after the fact, watermarking embeds an invisible signature into authentic media at the point of creation, allowing later verification of its origin and integrity. Combining watermarking with detection creates a more complete media authenticity ecosystem, one that is both proactive and reactive.

Conclusion of Future Directions

What is clear from all of the above is that this project does not conclude with its deployment — it begins there. The framework as it stands is capable, well-tested, and built on a technically sound foundation. But the landscape it operates in is dynamic, and the enhancements outlined here represent a considered and achievable roadmap for keeping it effective, expanding its reach, and deepening its usefulness across the full range of contexts where the authenticity of digital media matters.

Real-time capability, richer training data, multimodal analysis, mobile deployment, deeper explainability, and broader security applications — each of these is a meaningful step forward. Together, they represent a vision of a detection system that does not just respond to today's deepfakes, but is genuinely prepared for tomorrow's.

CHAPTER-09

PUBLICATION DETAILS

PUBLICATION DETAILS

Project Title: An Explainable Deep Learning Framework for Image and Video Deepfake Detection Using CNN-BILSTM-Vision Transformers

Authors: Dr. P. Kavitha, Chidadhala Venkatesh, VijayaSekarbabu N, Kathiravan S, A Mohammed Liyaakath, Subash Lenka

Paper Submitted to IEEE conference

CHAPTER-10

BIBLIOGRAPHY

10.1. REFERENCE

[1] Ian Goodfellow, J. Pouget-Abadie, M. Mirza, et al., "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[2] David E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, pp. 533–536, 1986.

[3] Sepp Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] Alex Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *NeurIPS*, 2012.

[5] Kaiming He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.

[6] Ashish Vaswani et al., "Attention Is All You Need," in *NeurIPS*, 2017.

[7] Alexey Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *ICLR*, 2021.

[8] Selvaraju Ramprasaath et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *ICCV*, 2017.

[9] Hany Farid, "Photo Forensics," MIT Press, 2016.

[10] Yuezun Li and S. Lyu, "Exposing DeepFake Videos By Detecting Face Warping Artifacts," in *CVPR Workshops*, 2019.

[11] Justus Thies et al., "Face2Face: Real-Time Face Capture and Reenactment of RGB Videos," in *CVPR*, 2016.

[12] Tero Karras et al., "A Style-Based Generator Architecture for Generative Adversarial Networks," in *CVPR*, 2019.

[13] Andreas Rossler et al., "FaceForensics++: Learning to Detect Manipulated Facial Images," in *ICCV*, 2019.

[14] Brian Dolhansky et al., "The Deepfake Detection Challenge Dataset," arXiv preprint arXiv:2006.07397, 2020.

[15] Hongguang Zhang et al., "Detecting Deepfake Videos with Temporal Inconsistencies," in *AAAI*, 2020.